

Designing with Dev-X: a systematic mapping of developer experience interventions and their business impact

Qayum, Abdul; Qureshi, Ayyaz; Razzaq, Abdul

Published in:
Information and Software Technology

DOI:
[10.1016/j.infsof.2026.108091](https://doi.org/10.1016/j.infsof.2026.108091)

Publication date:
2026

Document Version
Publisher's PDF, also known as Version of record

[Link to publication in ResearchOnline](#)

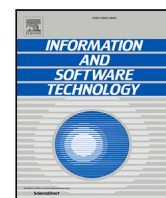
Citation for published version (Harvard):
Qayum, A, Qureshi, A & Razzaq, A 2026, 'Designing with Dev-X: a systematic mapping of developer experience interventions and their business impact', *Information and Software Technology*, vol. 195, 108091.
<https://doi.org/10.1016/j.infsof.2026.108091>

General rights

Copyright and moral rights for the publications made accessible in the public portal are retained by the authors and/or other copyright owners and it is a condition of accessing publications that users recognise and abide by the legal requirements associated with these rights.

Take down policy

If you believe that this document breaches copyright please view our takedown policy at <https://edshare.gcu.ac.uk/id/eprint/5179> for details of how to contact us.



Review article

Designing with Dev-X: A systematic mapping of Developer Experience interventions and their business impact

Abdul Qayum ^a, Ayyaz Qureshi ^b, Abdul Razzaq ^{a,*}

^a Department of Computer Science and Information Systems, University of Limerick, Ireland

^b Department of Computer Science, Glasgow Caledonian University, United Kingdom

ARTICLE INFO

Keywords:

Developer Experience
Dev-X interventions
Systematic mapping study
Business KPIs
Developer productivity

ABSTRACT

Context: Developer Experience (Dev-X) has emerged as a critical driver of software quality, developer productivity, and project success. However, the empirical literature remains fragmented, with limited understanding of how specific Dev-X interventions affect downstream software and business outcomes.

Objective: This study aims to synthesize and map the current empirical evidence on Dev-X interventions, identifying how they influence software outcomes (e.g., code readability, collaboration) and propagate to organizational key performance indicators (KPIs) such as product quality and process efficiency.

Method: We conducted a systematic mapping study of 160 empirical articles published between 2006 and 2024. We identified 146 unique interventions that impacts software outcomes such as role match may impact on developers' workload, likewise managing burnout in developer feedback may impact organizational culture. Using grounded coding, we also classified such mappings to 8 KPIs and 5 Dev-X dimensions based on the strength of empirical relationships (correlational, causal, classification-based) reported in the literature.

Results: Our analysis reveal that quality-related KPIs are more frequently impacted than productivity metrics, challenging traditional emphasis. Emotional state and values-oriented interventions dominate current practices, while cognitive load and motivation-focused strategies remain underexplored. Only a minority of studies (39 out of 160) explicitly trace full chains of intervention-outcome-KPI, highlighting persistent fragmentation; in other cases, we reconstructed these chains using cross-sectional synthesis of study data. Notably, recent years show an emergence of AI-assisted and value-centered intervention designs.

Conclusion: This study consolidates the fragmented Dev-X landscape by offering an evidence-based, open-access *Ready-Reckoner tool* for navigating interventions and their effects. While not prescriptive, it enables structured, context-aware exploration of empirical findings and lays the foundation for future causal, longitudinal, and multi-channel Dev-X research.

Contents

1.	Introduction	2
2.	Related work	3
2.1.	Conceptual foundations and framework evolution	3
2.2.	Dev-X interventions and their link to organizational KPIs	3
2.3.	Empirical gaps in Dev-X research and the case for structured mapping	3
3.	Methodology	4
3.1.	Research objectives and questions	4
3.2.	Search query formation	4
3.3.	Article search and selection	5
3.4.	Analysis, mapping, and validation	6
3.4.1.	RQ-driven analysis and inter-rater validation	8
3.5.	Selected literature characteristics	8
4.	Results	8
4.1.	RQ1: Mapping Dev-X facets, relationship types, and business KPIs	8

* Corresponding author.

E-mail addresses: abdul.qayum@ul.ie (A. Qayum), Ayyaz.Qureshi@gcu.ac.uk (A. Qureshi), abdul.razzaq@ul.ie (A. Razzaq).

4.2.	RQ2: Tracing interventions to software outcomes and KPIs.....	10
4.3.	Interactive ready-reckoner and replication package	10
5.	Discussion	14
5.1.	RQ1 and RQ2 findings: Practical and theoretical implications	14
5.2.	Recognized interventions and software outcomes: Practical mappings	15
5.3.	Longitudinal shifts in Dev-X intervention design and KPI focus	15
5.4.	Cross-facet and longitudinal gaps in Dev-X research	15
5.5.	Traceability and causality limitations	16
5.6.	Interactive ready-reckoner: What it does and how it helps	16
6.	Threats to validity	16
7.	Conclusion and future work.....	16
	CRedit authorship contribution statement	19
	Declaration of competing interest.....	19
	Appendix	19
	Data availability	19
	References.....	19

1. Introduction

Developer Experience (Dev-X), defined as the cognitive, emotional, and perceptual dimensions inherent to software development work [1], is increasingly recognized as a critical factor influencing both technical outcomes and organizational success. Traditional KPIs like productivity remain central to software engineering [2], but Dev-X extends this focus by influencing dimensions such as code quality, team dynamics, and process efficiency [3,4]. Empirical studies illustrate this breadth: elevated cognitive load during debugging tasks has been linked to reduced code comprehension and increased defect rates (e.g., [S1, S114]),¹ while unresolved emotional dissonance in distributed teams, such as frustration during code reviews, is linked to prolonged review cycles and diminished team cohesion [S137]. Similarly, alignment of developer values with organizational goals has been shown to enhance resource utilization and project success rates [S16, S88]. These findings position Dev-X as a mediator between individual developer efficacy and business-critical key performance indicators (KPIs) (e.g., productivity, process efficiency, project success, which serve as key indicators of organizational performance). Yet, despite growing recognition, research remains fragmented: few studies systematically construct directional chains from targeted Dev-X interventions, through intermediate software outcomes, to organizational KPIs as reported in empirical studies. This limits practitioners' ability to strategically select interventions that maximize business value. (For clarity, we define 'interventions' as empirically studied practices or strategies designed to influence specific Dev-X facets.)

Prior work has largely focused on identifying factors that influence Developer Experience (Dev-X) [4,5] or on isolated Dev-X facets – particularly cognitive load and productivity – without mapping how interventions propagate through software outcomes to business-relevant KPIs [2,3]. While existing frameworks catalog Dev-X improving practices [3,4,6], they rarely trace how these practices empirically relate to external outcomes or organizational impact. For instance, Greiler et al. [6] list 14 practices intended to improve Dev-X but do not map them to measurable KPIs. Similarly, facet-specific efforts (e.g., [7] for cognitive load; [8] for value alignment) seldom establish connections to downstream business outcomes such as project success or resource utilization [S43, S129].

This gap is further compounded by the limited exploration of longitudinal effects and cross-facet interactions. While interventions such as biometric monitoring for cognitive load [S116] and sentiment analysis for emotional feedback [S43] are well-documented, their long-term

impacts on KPIs like process efficiency or project sustainability are rarely empirically demonstrated [S16, S88]. Moreover, studies that investigate interactions between Dev-X facets – such as the interplay between cognitive load and developer values [S119, S160], or the influence of flow-enhancing interventions [S158] – remain scarce. Without such insights, practitioners struggle to design holistic strategies and cannot anticipate potential trade-offs or unintended consequences (e.g., interventions that improve flow might inadvertently increase cognitive load [S29]). As a result, the empirical landscape remains fragmented, offering little guidance on which interventions – such as sentiment-aware retrospectives [S43] or values-driven team alignment [S119] – are robustly associated with both software and organizational gains.

This paper addresses this gap by focusing on the **directional impact of targeted Dev-X interventions: how actionable strategies influence specific Dev-X facets to achieve desired software outcomes and business KPIs**. We conceptualize this directional logic – interventions manipulating Dev-X facets to influence software outcomes and business KPIs – as depicted in Fig. 1:

1. **Interventions** (e.g., sentiment analysis in Agile retrospectives [S43])
2. **Manipulate Dev-X Facets** (e.g., emotional states, cognitive load)
3. **Influence Software Outcomes** (e.g., debugging performance, code readability)
4. **Drive Business KPIs** (e.g., productivity, process efficiency).

Fig. 1 thus illustrates this causal chain and serves as a conceptual reference for tracing Dev-X impact across levels. For instance, enhancing perceived valence through positive feedback [S28] elevates developers' emotional states, which correlates with improved debugging performance (a quality outcome) and reduced defect rates (a productivity KPI). Similarly, reducing cognitive load via eye-tracking tools [S37] improves code readability, directly decreasing error density [S153]. Fig. 1 generalizes this logic, providing practitioners with a conceptual framework to trace how interventions propagate from individual experiences to organizational success.

Building on this conceptual model, we conduct a systematic mapping study [9] to map and analyze the relationships posited in Fig. 1. We systematically map 160 empirical studies (2006–2024), using a hybrid systematic mapping methodology [9,10], combining:

- **Taxonomy Development:** Classifying interventions (e.g., biometric monitoring [S116]), outcomes (e.g., code quality) for Dev-X facets, and software outcomes for KPIs (e.g., project success) using axial coding [11].
- **Relationship Coding:** Tagging study findings as *cause-effect*, *correlation*, or *qualitative association*, following guidelines for empirical software engineering [12].

¹ The 160 primary studies that we identified as the main dataset for our SLR are cited as [S1] to [S160]; all other literature is cited without an added 'S'.

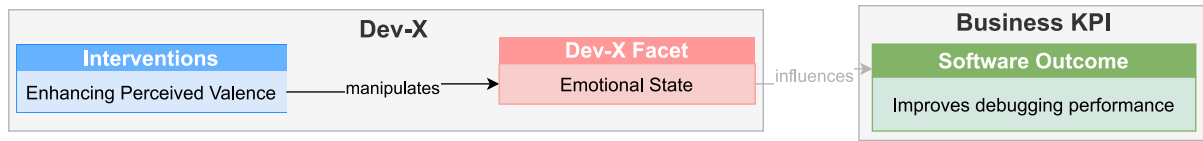


Fig. 1. Directional Flow from Dev-X Manipulation to Business Impact.

- **Gap Analysis:** Identifying underexplored pathways (e.g., values → flow [S84]) and longitudinal validation gaps (e.g., AI tools [S112]).

Our search strategy, validated via snowballing [13], spans ACM, IEEE, and Scopus, with inclusion criteria focusing on empirically measured Dev-X impacts (e.g., [S1]’s fMRI study on cognitive load). The resulting taxonomy and dataset, publicly available as a *ready-reckoner*, provide a reference resource to help practitioners explore Dev-X strategies reported in the literature, while guiding researchers towards high-impact gaps.

Contributions—This systematic mapping study advances the understanding of Developer Experience (Dev-X) by offering three key contributions:

1. **Evidence-Based Mapping:** *Synthesizing 160 studies to construct intervention → software outcome → KPI pathways, categorized by relationship strength.*
2. **Identification of Gaps and Implications:** *Highlighting underexplored intervention types (e.g., flow 6.9% of studies) and weakly validated causal pathways (e.g., long-term effects of AI-driven tools [S63, S112]). We also derive practical and theoretical implications to guide future Dev-X research and intervention design.*
3. **A Practitioner-Oriented “Ready-Reckoner”²:** *Delivering an open-access tool to support strategic, evidence-based Dev-X intervention planning.*

Organization—The remainder of this paper is organized as follows: Section 2 reviews foundational Dev-X frameworks and contrasts them with our systematic mapping approach. Section 3 details the research methodology, including search strategy, inclusion criteria, and data extraction protocols [9,10]. Section 4 presents the findings of our systematic mapping study, including the taxonomy of interventions and software outcomes. Section 5 discusses implications for research and practice, emphasizing underexplored relationships and methodological limitations. Section 6 addresses threats to validity and mitigation strategies. Finally, Section 7 concludes by summarizing key contributions and outlining future research directions.

2. Related work

This section situates our study within the evolving literature on Developer Experience (Dev-X), focusing on foundational models, empirical efforts connecting Dev-X interventions to business outcomes, and the need for structured synthesis. We highlight how our study extends prior work by systematically mapping Dev-X interventions, software outcomes, and business KPIs based on reported empirical evidence, without introducing new causal models.

2.1. Conceptual foundations and framework evolution

Developer Experience (Dev-X) emerged from efforts to treat developers as users within socio-technical environments, akin to the user experience (UX) paradigm. Fagerholm and Münch [1] introduced Dev-X as comprising cognition, affect, and conation—shaped by both internal and external influences. This conceptual foundation was later

enriched through psychological theories such as the Valence-Arousal-Dominance (VAD) model of affect [14,15] and Schwartz’s theory of basic human values [16], enabling operationalization of Dev-X facets, such as emotional states (via VAD models) and value alignment (via Schwartz’s framework).

Subsequent reviews expanded this conceptual base. Morales et al. [4] described Dev-X as a blend of productivity, satisfaction, and affective states, while Nylund’s multivocal review [5] incorporated practitioner literature to underscore Dev-X’s relevance in industrial contexts. Razzaq et al. [3] advanced this work by cataloging 41 Dev-X practices and associated productivity factors, though their study focused narrowly on productivity and did not model interactions across facets or software-to-business outcomes.

Table 1 summarizes the evolving landscape of Dev-X frameworks, highlighting each contributions and limitations.

2.2. Dev-X interventions and their link to organizational KPIs

Although conceptual models for Dev-X have matured, the empirical basis for connecting interventions to business outcomes remains fragmented. Storey et al. [2] linked job satisfaction to perceived productivity, while Greiler et al. [6] proposed actionable recommendations based on developer interviews. However, such studies rarely trace causal chains beyond isolated effects or extend findings beyond productivity-centric KPIs.

Domain-specific work follows a similar pattern. Gonçalves et al. [18] evaluated the cognitive load of developers during comprehension tasks, and Hussain et al. [19] examined human values in software practices. These studies focus on individual Dev-X facets but do not connect them to downstream software quality or team dynamics. Tool-based interventions – such as sentiment-aware retrospectives [S43], biometric tracking [S116], and LLM-based assistants [S112] – have demonstrated promise in isolated contexts. However, their role within broader Dev-X strategies linked to project success or resource efficiency remains underexplored.

2.3. Empirical gaps in Dev-X research and the case for structured mapping

Three persistent gaps can be identified in the current Dev-X literature.

First, empirical studies rarely trace the full directional pathway from Dev-X intervention to software outcomes and business KPIs. Most existing research centers on single-facet links (e.g., emotional state → satisfaction) or assumes productivity as the primary outcome [2,3], leaving broader constructs like process efficiency or team cohesion understudied.

Second, prior reviews offer little insight into cross-facet relationships or longitudinal dynamics. For instance, few studies examine how values and emotional states together shape developer flow [S84], or how interventions such as AI-assisted code summarization influence project sustainability over time [S112]. Furthermore, cause-effect relationships – critical for supporting intervention selection – are underrepresented, with most studies relying on correlational or qualitative evidence [4,6].

Third, although a range of Dev-X support tools exist – such as EmoVizPhy for emotion feedback [S43] or SentiStrength-SE for textual sentiment analysis [S136] – they are often isolated prototypes and not integrated into broader organizational strategies. Practitioners

² <https://ready-reckoner-devx.netlify.app/>

Table 1
Related work landscape: focus areas and limitations.

Study	Focus areas	Limitations
Fagerholm & Münch [1]	Cognition, affect, conation	No empirical mapping to business KPIs
Storey et al. [2]	Satisfaction ↔ productivity	Narrow KPI lens; no intervention chains
Morales et al. [4]	Productivity, satisfaction, affect blend	Limited cross-facet/KPI structuring
Greiler et al. [6]	Actionable factors, barriers	Generalizability; no quantitative chains
Razzaq et al. (2024) [3]	Practices and productivity factors	Productivity-centric; lacks KPI breadth
Nylund [5]	Practitioner Dev-X perspectives	Grey sources; no KPI mapping
Razzaq et al. (2025) [17]	Empirical state methods-data-metrics metric corroboration	Focuses on assessment design no intervention→outcome→business-KPI chains limited KPI traceability
This Study (2026)	Facet × Relationship × KPI Intervention→Outcome→KPI ready-reckoner tool	Causal evidence limited some KPIs underrepresented

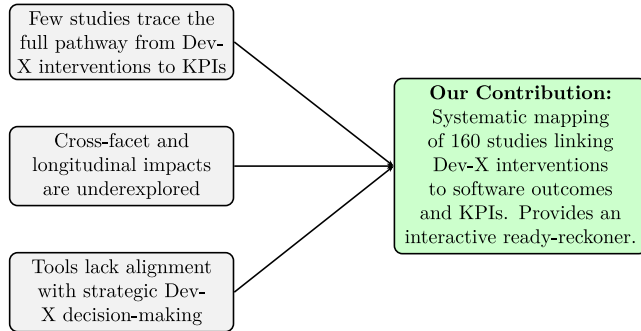


Fig. 2. Summary of gaps in Developer Experience (Dev-X) literature and this study's contributions.

thus lack structured, empirically grounded references to guide Dev-X interventions in alignment with project-level goals.

Our Contribution—This study addresses these gaps through a systematic mapping of 160 empirical articles, synthesizing Dev-X interventions, their targeted facets, the software outcomes they influence, and their alignment with business KPIs. As visualized in Fig. 2, our work bridges the conceptual and empirical divide by (1) categorizing relationships by empirical strength (e.g., causal, correlational), and (2) delivering a practitioner-oriented ready-reckoner tool to support evidence-based intervention planning.

3. Methodology

This section outlines the rigorous, replicable methodology adopted in our systematic mapping study.

We conduct a *Systematic Mapping Study (SMS)* following Petersen et al. [9]. Accordingly, we (i) formulate search query and pilot a search strategy, (ii) perform an article search and selection with predefined criteria, (iii) extract, code, and analyze data using an agreed taxonomy, and (iv) map findings to reveal concentrations and gaps. We also cite Kitchenham and Charters [10] for general SLR rigor where relevant to screening and reporting, and Wohlin [13] for snowballing. Each step was designed to ensure coverage, transparency, and analytical robustness.

3.1. Research objectives and questions

This study aims to empirically synthesize and structure the current knowledge on Developer Experience (Dev-X) interventions, the facets they target, and their reported impact on software and business outcomes. We seek to enable researchers and practitioners to trace the directional influence of interventions – such as reducing cognitive load – on development outcomes (e.g., code comprehension) and organizational KPIs (e.g., productivity, process efficiency).

To guide this synthesis, we pose the following research questions:

- **RQ1 (Facets→ KPI):** Which Dev-X facets have been examined in the literature, and how do they link with business-critical KPIs such as project success?

How addressed (RQ1). For each study we extract {Facet, Relationship Type, KPI} triplets; e.g., *motivation→correlation→productivity, cognitive load→cause-effect→project success*.

- **RQ2 (Interventions→ Outcomes/KPI):** What targeted manipulations (interventions) addressed the specific Dev-X facets, and how do these manipulations impact external software outcomes to support reported improvements in business KPIs?

How addressed (RQ2). For each study we extract directional chains {Intervention → Outcome → KPI}; e.g., *LLM summarization → code comprehension/debugging accuracy → Quality; empathy-improving practices → user-developer empathy ↑ → Team Dynamics; HRM practices (developmental appraisal, equitable reward, training) → completion likelihood ↑ → Project Success*.

Fig. 3 illustrates our complete methodology: from search query design and article selection across major digital libraries, to screening, coding, and the RQ-specific synthesis strategies. The process culminates in a structured evidence-based mapping used to generate our ready-reckoner tool.

3.2. Search query formation

To ensure broad and replicable coverage of empirical literature on Developer Experience (Dev-X), we constructed a comprehensive, evidence-informed search strategy aligned with systematic mapping guidelines [9,10] and snowballing best practices [13]. Our approach leveraged both prior literature and iterative refinement to balance coverage with precision.

Existing SLRs and Seed Articles: We began by analyzing four prominent secondary studies on Dev-X—specifically, prior SLRs and mapping efforts by Razzaq et al. [3], Greiler et al. [6], Morales et al. [4], and Nylund [5]. These served as seed papers and provided the initial vocabulary of Dev-X facets (e.g., motivation, cognitive load), outcomes (e.g., productivity, quality), and KPIs (e.g., team dynamics, project success). We also extracted keywords from 36 additional empirical articles referenced in these seed reviews. We analyzed the *full text* of these four seed studies and 36 referenced empirical papers to derive and validate the initial vocabulary.

Corpus Analysis. We mined unigrams and bigrams from the *full text* of the 40-paper seed corpus (4 secondary studies + 36 empiricals). Terms were filtered with a curated English stopword list (derived from NLTK) plus domain boilerplate removal, and normalized for hyphenation (e.g., *well-being*). We retained domain terms using document-frequency thresholds (unigrams $df \geq 5$, bigrams $df \geq 3$) and curated expansions (aliases/synonyms for Dev-X, facets, and KPIs). To mitigate coverage bias, we balanced umbrella labels (*Developer Experience, Dev-X, DevX*) with facet- and KPI-specific terms (e.g., *cognitive load, process efficiency, flow*). Search terms were reviewed and refined in multiple

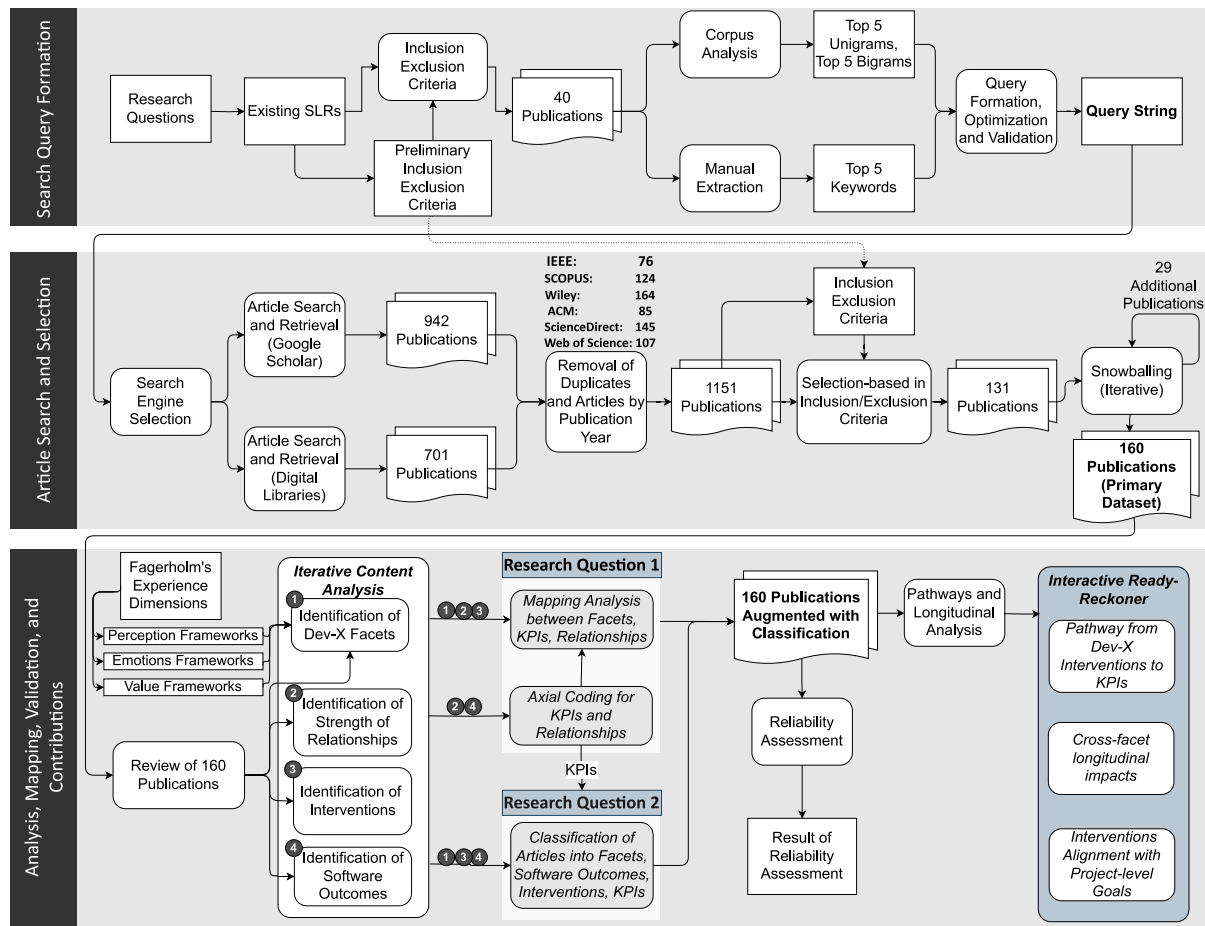


Fig. 3. Systematic mapping methodology pipeline driven by research questions.

iterations with author consensus.³ Also, full-text n-gram statistics used to refine the vocabulary are provided with processing scripts.

Query String (Google Scholar syntax):

```
((‘Developer Experience’ OR ‘Dev-X’ OR ‘DevX’ OR Developer) AND (‘Experience’ OR ‘Motivation’ OR ‘Cognitive Load’ OR ‘Emotions’ OR ‘Values’ OR ‘Satisfaction’)) AND (‘Outcomes’ OR ‘Quality’ OR ‘Productivity’ OR ‘Success’) AND (‘Project’ OR ‘Code’ OR ‘Organization’ OR ‘Software’)
```

Note: In structured databases (ACM/IEEE/Scopus), we used fielded queries and proximity operators to restrict ambiguous aliases (e.g., NEAR/3 linking *developer* and *experience*).⁴

3.3. Article search and selection

Search Engine Selection: We applied the query across eight major digital libraries and indexing engines – spanning computer science, software engineering, and HCI domains – between August and October 2024. To ensure breadth, we used both domain-specific (e.g., ACM, IEEE) and multidisciplinary (e.g., Scopus, Google Scholar) databases. Google Scholar was used as a supplementary source to capture empirical studies not indexed in curated libraries:

• *ACM Digital Library, IEEE Xplore, SpringerLink, ScienceDirect (Elsevier), Scopus, Web of Science, Wiley Online Library, Google Scholar*

The base query was customized per database (e.g., field-specific syntax) to maintain compatibility while preserving its semantic scope. In all cases, searches were limited to peer-reviewed English-language articles.

Article Search and Retrieval: We aggregated results from all databases into a unified corpus, resulting in **1,643 raw records**. Duplicates were removed using metadata matching (title, author, year, DOI), yielding **1,151 unique articles**. This curated corpus was retained for downstream screening. A detailed search log – including database-specific queries and retrieval counts – is available in the replication package.⁵

Inclusion Criteria Articles were retained if they satisfied all of the following conditions:

- **Empirical Evaluation of Dev-X Facets:** The study must empirically investigate at least one Developer Experience (Dev-X) facet – such as cognitive load, motivation, emotional states, or values – using quantitative, qualitative, or mixed methods.
- **Link to Software Outcomes or Business KPIs:** The study must report a relationship between a Dev-X facet and either software outcomes (e.g., debugging performance, tool usability) or organizational KPIs (e.g., productivity, team cohesion, resource utilization).

³ The results for searches on ACM/IEEE/Scopus/Google Scholar and adjustments are logged in the replication package in Coverage Analysis.

⁴ Exact per-database strings and filters are listed in the replication package.

⁵ Replication Package: <https://doi.org/10.5281/zenodo.17799974>

- **Actionable Relevance:** The study must provide empirical insight into an intervention or manipulation (e.g., a practice, strategy, or tool) that impacts Dev-X. Evidence must be reported through causal, correlational, predictive, or qualitative findings.
- **Scientific Rigor and Scope:** The article must be peer-reviewed and published between **October 2006 and October 2024**.

Rationale for Inclusion Criteria: These criteria were chosen to ensure the dataset captures studies with actionable and empirically supported insights relevant to both Dev-X and software/business impact. Articles without measurable Dev-X facets or lacking downstream outcomes were excluded to maintain alignment with our research questions. Furthermore, we excluded papers that only referenced tools or environments without assessing their influence on Developer Experience or related performance metrics.

Exclusion Criteria Articles were excluded if they met any of the following:

- **Non-empirical nature:** Essays, opinion pieces, theoretical models without empirical validation.
- **Tool-centric without Dev-X focus:** Papers focused solely on tool performance, scalability, or architecture without assessing Dev-X implications.
- **Non-developer user experience:** Studies that addressed customer or end-user experience without any relevance to internal developer-facing aspects.

Time Window Justification (2006–2024): We selected **October 2006** as the lower bound to capture early empirical studies on developer cognition, affect, and values—predating but informing the formal definition of Dev-X by Fagerholm and Münch in 2012 [1]. This decision ensures inclusion of foundational work that shaped the conceptual landscape of Dev-X. The upper bound of **October 2024** reflects the most current body of peer-reviewed research available at the time of conducting this study.

Remove of Duplicates and Selection based on Inclusion/Exclusion Criteria: Following the deduplication of 1151 retrieved records, we initiated a structured screening phase. Each article was independently reviewed by two researchers to assess its relevance based on predefined inclusion and exclusion criteria. Reviewers were instructed to evaluate the title and abstract, and refer to the full text when the decision could not be made with confidence. If both reviewers agreed, the article was included or excluded accordingly. In the case of disagreement (18 papers), a third reviewer made the final decision through independent assessment. This initial process yielded **131 articles**.

Snowballing (Iterative): To ensure broader coverage and mitigate the limitations of keyword-based search (e.g., indexing or vocabulary mismatches), we complemented database searches with snowballing as recommended by Wohlin [13]. We first performed backward snowballing on a diverse set of 10 seeds selected by (i) topical centrality to Developer Experience (Dev-X), (ii) diversity of venue, year, and method, and (iii) citation influence. This seed set size (10) was chosen to keep screening effort tractable while preserving topical breadth, in line with snowballing guidance that recommends a manageable yet diverse set of seed studies [13]. Backward snowballing Round R1 yielded 29 additional included studies after deduplication and screening. We evaluated a second backward iteration (R2) using the R1 inclusions as seeds under a pre-specified stopping rule (terminate when a round yields no new eligible studies after screening); R2 produced no new inclusions, so we stopped.

To verify recency coverage, we also conducted a forward-citation check on Scopus and Web of Science using the same seed set within our time window. Consistent with our database results and our breadth check on Google Scholar (942 raw results), the forward check did not surface additional eligible studies beyond duplicates/out-of-scope items. Full per-round logs and the stopping rule are provided in the

replication package. Snowballing contributed an additional 29 papers, yielding a final dataset of **160 primary empirical studies**.

Coverage Analysis: To assess coverage bias, we ran three query variants per source—*Base* (concept + facet/KPI block), *Agnostic* (concept-only), and *Alias-only* (Dev-X/DevX/DevEx labels, explicitly excluding ‘developer experience’). We executed these on ACM, IEEE, and Scopus (metadata vs full-text scope recorded) and used Google Scholar for Top-K overlap. The Agnostic variant consistently expanded recall without adding exclusive screened-in evidence; the Alias-only variant returned a tiny, recent set.⁶

3.4. Analysis, mapping, and validation

To answer our research questions, we employed a structured data extraction and classification procedure grounded in established synthesis guidelines for empirical software engineering [10,13]. We adopted the **Grounded Theory** approach, particularly the *axial coding* methodology of Corbin and Strauss [11], which enabled us to iteratively abstract conceptual categories from diverse empirical findings.

This method was especially appropriate for Developer Experience (Dev-X) research, which spans psychological constructs, developer behavior, and organizational goals. Axial coding supported the consolidation of raw observations (e.g., perceived difficulty, debugging time) into higher-level themes (e.g., cognitive load, productivity, or code quality), and enabling a traceable, theory-informed synthesis.

Data Extraction Dimensions. We reviewed each of the 160 primary studies in full-text, extracting the following dimensions:

- **Dev-X Facet:** The focal psychological or perceptual construct (e.g., cognitive load, emotional states, motivation, values), based on frameworks such as Valence-Arousal-Dominance [14], Parrott’s emotion taxonomy [20], and Schwartz’s value theory [16]. [Table 2](#) presents the definitions and operational cues, as well as examples for each Dev-X facet.
- **Intervention:** Any explicitly reported manipulation, strategy, or tool designed to affect a Dev-X facet (e.g., retrospectives, biometric tracking, LLM-based summarization).
- **Software Outcome:** Observable impacts at the developer or artifact level (e.g., code readability, debugging accuracy, developer flow).
- **Business KPI:** Organizational metrics influenced by outcomes, derived through axial coding of study-reported indicators (e.g., productivity, process efficiency, team dynamics). Their typical indicators (as found in literature), along with the definition and examples, are presented in [Table 3](#).
- **Relationship Strength:** We classified links into five mutually exclusive categories as defined using the following explicit design/analysis cues, which are exemplified using selected papers⁷:

1. **Correlation/Association**—associations reported without manipulation, e.g., developers’ perceived valence/arousal co-varied with self-reported flow [S2];
2. **Cause-Effect**—directional effects supported by an intervention or treatment, e.g., inducing high arousal via film clips improved debugging performance [S3];
3. **Comparison**—group/condition differences without explicit identification, e.g., teams emphasizing emotional-intelligence practices reported stronger trust than comparison groups [S14];

⁶ Exact query strings, filters, and retrieval deltas are archived in the replication package.

⁷ Traceability to the evidence related to relationship strength can be found in *Data Analysis* sheet in replication package.

Table 2

Dev-X Facets—definitions, operational cues, examples, with representative study IDs. Full mapping in the replication package.

Facet	Definition & operational cues	Example observable	Representative studies (IDs)
Cognitive Load	Mental effort during SE tasks; cues: NASA-TLX, comprehension accuracy/time, interruption lag, EEG/fNIRS/EDA, eye-tracking.	Code understanding correctness; task time; perceived difficulty.	S1, S6, S7, S23, S37, S47, S53, S81, S102, S110, S113, S114, S115, S118, S142, S153, S154, S155, S126
Motivation	Goal/need fulfillment and engagement (e.g., intrinsic motivation, autonomy, meaning). Cues: motivation scales, deployment/participation behaviors.	Perceived productivity; retention/willingness; deployment frequency.	S8, S15, S20, S25, S30, S31, S52, S69, S86, S95, S101, S105, S108, S128
Emotional States	Valence-Arousal-Dominance and discrete emotions; cues: affect grids/PANAS, sentiment/lexicons, mood induction, physiological proxies.	Happiness/valence; frustration; emotional intensity effects.	S2, S3, S4, S5, S10, S11, S12, S22, S24, S27, S28, S38, S39, S49, S54, S56, S58, S61, S65, S67, S68, S70, S71, S72, S73, S75, S76, S77, S78, S80, S89, S90, S94, S134, S136, S137, S138, S139, S144, S147, S150
Values	Value alignment/ownership/ethics/culture influencing work; cues: value inventories, ownership, trust/cohesion, prioritization, diversity.	Commitment/ownership; trust; value congruence; ethical practice.	S16, S32, S35, S36, S42, S46, S55, S56, S96, S97, S99, S100, S103, S109, S112, S117, S119, S121, S124, S127, S130, S135, S140, S148, S149, S156, S157, S160
Other	Signals not squarely in the above (e.g., personality, episodic DX, generic well-being composites, measurement instruments).	Personality effects; episodic DX; bespoke instruments.	S9, S17, S34, S41, S57, S79, S83, S85, S133, S141, S145

Table 3

Business KPIs—definitions, typical indicators, examples, with representative study IDs. Full mapping in the replication package.

KPI	Definition & typical indicators	Example observable	Representative studies (IDs)
Productivity	Effective output per unit time; perceived productivity; throughput/velocity. Indicators: commits, tasks completed, cycle time, survey ratings.	More commits; shorter task time; higher perceived productivity.	S1, S3, S10, S11, S15, S16, S19, S20, S21, S25, S28, S31, S37, S38, S39, S47, S52, S54, S55, S56, S57, S58, S59, S70, S73, S75, S76, S83, S85, S86, S87, S88, S94, S101, S105, S108, S120, S128, S133, S145
Process Efficiency	Lead/cycle time, rework/hand-offs, WIP, adherence to process; lean flow efficiency.	Shorter PR/lead time; fewer hand-offs; smoother throughput.	S11, S16, S31, S52, S57, S82, S92, S100, S104, S117, S128, S160
Quality	Internal/external quality: defects, smells, readability, review rework, static warnings.	Lower defect density; improved readability; less rework.	S1, S6, S7, S11, S12, S17, S23, S32, S33, S34, S40, S44, S45, S47, S49, S51, S53, S54, S57, S60, S61, S65, S68, S71, S72, S78, S80, S81, S82, S86, S92, S96, S102, S110, S113, S114, S115, S116, S122, S123, S125, S126, S136, S137, S138, S139, S140, S141, S142, S143, S146, S148, S149, S152, S153
Team Dynamics	Trust, cohesion, coordination, conflict, retention risk, culture.	Higher trust/cohesion; fewer conflicts; better coordination.	S12, S14, S42, S46, S67, S74, S91, S95, S96, S100, S103, S108, S117, S119, S121, S124, S127, S130
Flow	Psychological flow as an outcome/KPI (absorption, reduced fragmentation) and work-pattern signals.	Higher flow ratings; fewer interruptions/context switches.	S2, S21, S24, S29, S62, S82, S84, S101, S106, S118, S158
Resource Utilization	Efficient use of developer time/capacity or tooling resources; fragmentation and multitasking costs.	Less context switching/rework; better capacity use.	S15, S64, S86, S111, S144, S147, S150, S151, S157, S159
Developer Well-being	Burnout, stress, recovery, psychosocial risk and satisfaction.	Lower burnout/stress; improved well-being scores.	S18, S26, S41, S55, S56, S58, S74, S77, S85, S93, S95, S100, S104, S109, S128, S133
Project Success	Delivery success/sustainability and stakeholder satisfaction/retention.	On-time delivery; sustained activity; better stakeholder ratings.	S8, S13, S40, S46, S50, S52, S59, S63, S67, S84, S92, S101, S112, S123, S131, S135

- Prediction/Classification**—models predicting outcomes with out-of-sample validation, e.g., emotion-word features predicted bug severity classes [S80];
- Qualitative Association**—relationships inferred from qualitative analyses, e.g., reduced morale was linked to increased technical debt and culture impacts [S12].

Axial Coding and Open Card Sorting (RQ2). Our classification process involved four stages:

- Open Coding:** We identified and annotated fine-grained descriptors (e.g., “reduced bug resolution time”, “enhanced code clarity”) across all studies.
- Axial Coding (RQ1 baseline):** Codes were clustered into higher-level categories across three axes – Dev-X facets, business KPIs, and relationship types – based on inter-conceptual relationships.

- Open Card Sorting (RQ2 grouping; Spencer [21]):** To derive interpretable intervention groupings for RQ2, two coders independently grouped minimally phrased “cards” (interventions and outcomes), then reconciled category names and boundaries while recording rationale. Inter-coder agreement was assessed using Cohen’s κ per pair; categories were refined when $\kappa < 0.60$. The *Card Sorting Protocol* and template are included in the replication package.
- Consensus Validation:** All clustered categories were reviewed and iteratively refined through group consensus to ensure semantic clarity and orthogonality.

This combined inductive procedure (axial coding + card sorting) moves beyond surface-level tagging to reveal structured patterns grounded in empirical evidence. The resulting framework underpins the analyses in Sections 4 and 5 and is summarized in Fig. 4.

Quality Assurance and Reproducibility. To ensure consistency and replicability, we developed a controlled vocabulary and extraction

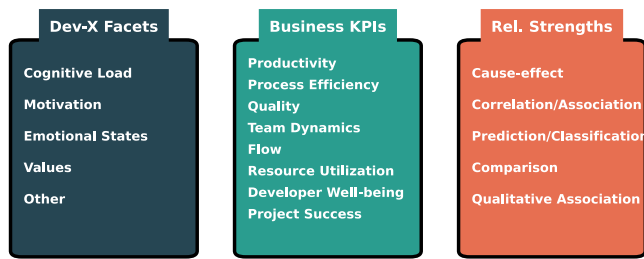


Fig. 4. Final taxonomy of Dev-X facets, KPI categories, relationship types (RQ1), and RQ2 intervention groupings derived via axial coding and open card sorting.

handbook, iteratively refined in a pilot phase (16 articles). Coding disagreements were reconciled through discussion and calibration. Inter-coder agreement for the card-sort stage was monitored with Cohen's κ ; categories failing $\kappa \geq 0.60$ were revisited. Final documentation (including the Card Sorting Protocol and template), the complete classification schema, and the dataset are available in our open-access replication package.⁸

3.4.1. RQ-driven analysis and inter-rater validation

Our analysis followed the structure and categories derived from axial coding (Section 3.4), aligning with each research question's objective.

Analysis for RQ1: Mapping Dev-X Facets, Relationship Types, and KPIs. We extracted structured triplets from each article in the form:

{Dev-X Facet, Relationship Strength, Business KPI}
These triplets allowed us to perform:

- **Frequency and co-occurrence analysis** to reveal dominant themes and overlooked connections (e.g., emotional states → team dynamics).
- **Gap identification** by analyzing sparsely represented facet-KPI-relationship combinations (e.g., values → process efficiency via cause-effect).

Analysis for RQ2: Tracing Interventions to Software and Business Outcomes. For RQ2, we filtered the dataset to include only articles explicitly reporting a Dev-X intervention. From each, we extracted a directional pathway:

{Intervention → Dev-X Facet → Software Outcome → Business KPI}
This pathway enabled us to:

- **Group interventions** into mechanism-based clusters (e.g., biometric sensing, emotion tracking, AI-assisted tools).
- **Trace outcome chains** that linked intervention impact from the individual level (e.g., debugging accuracy) to the organizational level (e.g., project success).
- **Identify high-utility mappings** that demonstrate a complete link from intervention to KPI via Dev-X facet and software outcome.

These two levels of structuring – Dev-X facet mappings for RQ1 and intervention-driven outcome pathways for RQ2 – allow fine-grained analysis of both experience-outcome and strategy-outcome links (see Sections 4 and 5). They also support the development of a practitioner-oriented ready-reckoner that enables real-world lookup of Dev-X strategies based on empirical strength and relevance.

Reliability of Classification. To ensure the validity of subjective assignments – such as Dev-X facet identification, KPI mapping, and relationship strength classification – we conducted a formal inter-rater reliability assessment on a stratified sample of 16 studies. Two

coders independently assigned categories using the schema developed in Section 3.4. Discrepancies were reviewed and resolved through consensus.

The overall agreement yielded a **Fleiss' Kappa score of $\kappa = 0.79$** , indicating *substantial agreement* according to standard thresholds [22]. This high agreement reinforces the reliability of our thematic assignments and the validity of our mapping framework.⁹

3.5. Selected literature characteristics

The final dataset comprises **160 primary studies** published between **2006 and 2024**, covering a diverse range of empirical Dev-X research. Fig. 5 presents the publication trends over time, categorized by venue type. The analysis reveals a consistent growth in scholarly attention to Developer Experience, particularly in journal publications since 2017. The volume peaks in 2017 ($n=19$), with a secondary high in 2023 ($n=17$), with a notable contribution across journals (J), conferences (C), and other scholarly venues (O), such as workshops and short papers.

As shown in Fig. 5, the overall distribution is relatively balanced across journals (73) and conferences (80), with a small number of publications in other venues (7). However, in the more recent years, journals become the predominant outlet, indicating a shift towards more mature and rigorously developed empirical work in the field.

We also analyzed the **citation distribution** across the 160 studies. As depicted in the top-left inset of the figure, a majority (**103 studies**) had citation counts of ≤ 50 , while a smaller subset (**57 studies**) exceeded 50 citations, indicating both recentness and emerging impact of Dev-X research. On average, each article is cited for **59.63** (as of February 2024) times as per GoogleScholar citation count. This distribution supports our interpretation that the field is active but still evolving, with significant potential for future high-impact contributions.

Summary. By grounding our extraction and analysis in established systematic mapping guidelines and axial coding procedures, we ensured both methodological rigor and practical relevance. The resulting dataset of 160 studies was analyzed to uncover Dev-X interventions, their targeted facets, and their reported impact on software and organizational outcomes—forming the foundation of the results presented in the next section.

4. Results

4.1. RQ1: Mapping Dev-X facets, relationship types, and business KPIs

To address **RQ1**, we analyzed 160 empirical studies to determine which Developer Experience (Dev-X) facets have been studied [1,2,4], the types of relationships reported, and the business KPIs impacted. Each study was coded into one or more structured triplets of the form:

{Dev-X Facet, Relationship Strength, Business KPI}

Fig. 4 presents the final coding taxonomy used for our classification, while Fig. 6 visualizes the frequency and distribution of Dev-X facet, KPI, and relationship type combinations across the 160 studies. This mapping reveals the most empirically studied Dev-X facet-KPI-relationship intersections and highlights critical underexplored areas.

Dev-X Facets (see Table 2 for definitions and operational cues): The most frequently analyzed facet was **Emotional States** (94 instances), followed by **Values** (65), **Cognitive Load** (35), **Motivation** (18), and a residual **Others** category (12). Notably, common facets such as *emotional states* (58.8%) and *values* (40.6%) were linked to a wide range

⁸ <https://doi.org/10.5281/zenodo.17799974>

⁹ The Reliability Assessment folder in the Replication Package presents the reviewed paper set and results

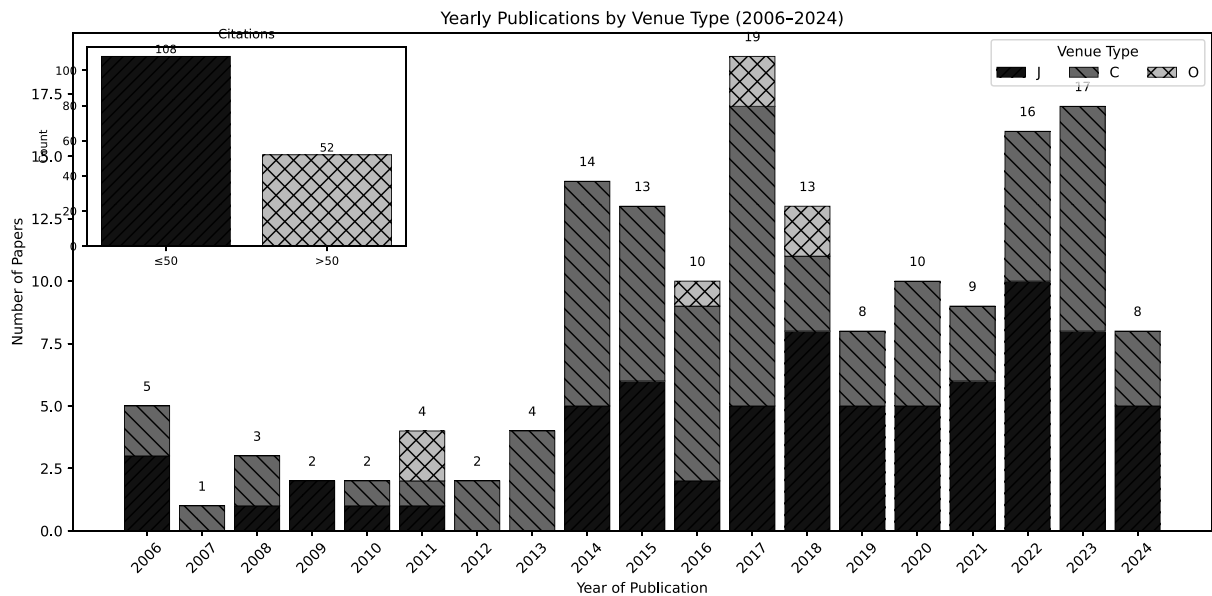


Fig. 5. Publication trends (2006–2024) by venue type, showing peak interest in 2023 and growing journal-based maturity.

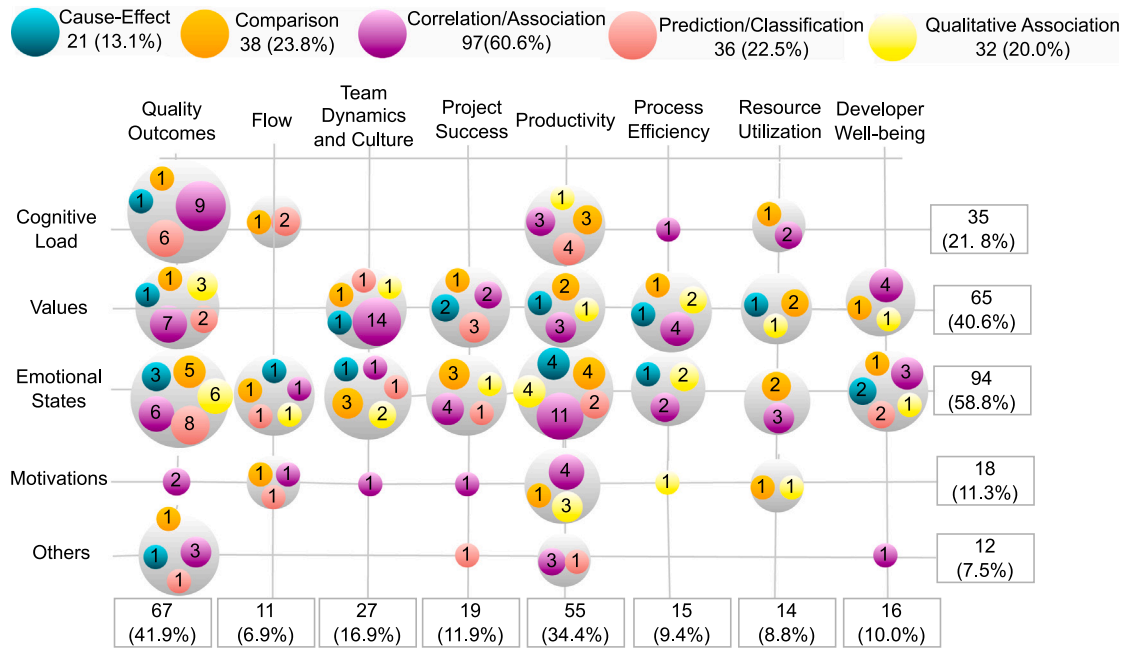


Fig. 6. Mapping between Dev-X Facets, KPIs, and Types of Relationships. Our online ready-reckoner resource allows the traceability of the results presented in this Figure: <https://ready-reckoner-devx.netlify.app/>.

of outcomes [5,6]; however, values did not appear to influence the flow state in any of the reviewed studies.

Business KPIs (see Table 3 for definitions and typical indicators): The most frequently reported KPI was **Quality Outcomes** (67), followed by **Productivity** (55), **Team Dynamics and Culture** (27), and **Project Success** (19). Less frequently targeted KPIs include **Developer Well-being** (16), **Process Efficiency** (15), **Resource Utilization** (14), and **Flow** (11). The coverage sensitivity check (Base vs Agnostic vs Alias-only) indicates this under-representation is not an artifact of the base query formulation. These findings nuance the common emphasis on productivity by showing quality outcomes are at least as prominent in the mapped evidence. In fact, **quality outcomes** – linked in 41.9% of studies – were the most commonly impacted dimension, surpassing **productivity** (34.4%) [2,3]. This reflects a broader understanding of Dev-X as a driver of engineering practice improvements beyond output metrics. Many studies linked their findings to multiple KPIs, highlighting the multidimensional influence of Dev-X.

Relationship Strengths: Most studies reported **Correlational or Associational** relationships (97), indicating reliance on survey and observational methods. **Comparison** (38) and **Prediction/Classification** (36) relationships were also common, often relying on quasi-experiments or ML-based analyses. Only 21 studies reported **Cause-Effect** relationships, underscoring a methodological gap in rigorous, intervention-driven validation of Dev-X strategies. **Qualitative Associations** (32) appeared frequently in interpretive studies such as case studies and interviews.

Together, these findings suggest a conceptual and methodological skew in the literature: (1) emotional and value-based Dev-X facets dominate over cognitive or motivational aspects; (2) KPIs such as *Flow*, *Process Efficiency*, and *Resource Utilization* are underrepresented; and (3) causal evidence remains limited. These results provide a foundation for identifying high-impact but under-researched Dev-X–KPI relationships, which we explore further in RQ2 and the discussion. Percentages may exceed 100.¹⁰

4.2. RQ2: Tracing interventions to software outcomes and KPIs

To address RQ2, we analyzed how empirical studies link specific interventions – such as tools, strategies, or practices – to Developer Experience (Dev-X) facets, the resulting software outcomes, and corresponding business KPIs [10,13]. These directional pathways are represented as:

{Intervention → Dev-X Facet → Software Outcome → Business KPI}

The 160 primary studies described Dev-X interventions along with their downstream effects on software outcomes or KPIs, enabling partial or full traceability. These mappings form the foundation of our synthesized intervention-outcome-KPI mapping, presented in Table 4, which aggregates all linkages between interventions, targeted Dev-X facets, software outcomes, and business KPIs.

Software Outcomes: Table 4 aggregates directional mappings from 160 studies, using short-form KPI codes (e.g., Prod. for Productivity, Qual. for Quality, Flow for Flow state), which are defined in Fig. 4. The most frequently observed software outcomes include **debugging performance** [S54, S89], **code readability** [S110, S113], **developer flow** [S158], **collaboration** [S137], and **tool usability** [S23]. Several studies demonstrated how interventions – such as valence/arousal regulation [S4, S44] or eye-tracking-based cognitive load reduction [S116, S118] – contributed to measurable improvements in these outcomes, thereby facilitating improved developer effectiveness and coordination.

Business KPIs Impacted: Among the mapped interventions, the most frequently associated KPIs were **Productivity** [S31, S107], **Quality**

[S153, S160], and **Team Dynamics** [S43, S119]. Others such as **Developer Well-being** [S109], **Flow** [S158], and **Process Efficiency** [S43] were also represented but less prominently. Interventions rooted in value alignment [S84, S128] or ethical considerations [S66] tended to connect with longer-term KPIs such as *project success* [S160] and *resource utilization* [S84].

Traceability and Completeness: Out of 160 studies, only 39 provided a fully traceable chain from intervention to software outcome and business KPI within the article itself. In the remaining 121 studies, Dev-X manipulations were reported to affect either a Dev-X facet or intermediate software outcome, but lacked a direct link to higher-level KPIs. In such cases, we inferred the likely KPI based on known constructs (e.g., debugging efficiency → quality). This gap presents a critical opportunity for future research to develop end-to-end validations of Dev-X strategies, particularly those aimed at business-level KPIs.

Illustrative Highlights: Select examples from Table 4 include:

- **Cognitive Load:** Eye-tracking and psychometric tools [S115, S116, S1, S37] improved comprehension [S110, S113], usability [S23], and focus [S47]; excessive load increased difficulty [S6, S29].
- **Emotional States:** Mood regulation [S4, S44], affect-tagged commits [S65], and retrospective feedback [S43] enhanced debugging, collaboration, and process efficiency.
- **Motivation:** Reflection-based goal setting [S107] and early deployments [S20] improved engagement and developer habits [S31].
- **Values:** Diversity-aligned strategies [S119, S128, S160] and ethics-aware practices [S66] promoted retention, cohesion, and project alignment [S129].
- **Tool Support:** LLM-powered summarizers [S112], fatigue-reduction dashboards [S109], and emotion-tagged bots [S58] supported comprehension and emotional regulation.

Overall, our findings demonstrate a growing diversity in Dev-X interventions and their empirical grounding. Yet, they also reveal a persistent reporting gap—particularly in linking Dev-X strategies to long-term organizational KPIs through explicit traceable pathways.

4.3. Interactive ready-reckoner and replication package

To support the practical application of our findings and enhance transparency, we developed an interactive **Dev-X Ready-Reckoner Tool**: <https://ready-reckoner-devx.netlify.app/>. This online resource allows researchers and practitioners to explore the empirical mappings captured in our study – linking *Dev-X interventions, facets, software outcomes*, and *business KPIs* – across 160 reviewed articles.

Features of the Ready-Reckoner:

- **Faceted Search Interface:** Users can filter by Dev-X facet, intervention type, KPI, or relationship strength.
- **Evidence Mapping:** Each entry is linked to the original study (e.g., [S43], [S112]) along with the reported relationship type (e.g., cause–effect, correlation).
- **Practical Insights:** The tool highlights outcomes influenced by each intervention, allowing context-specific exploration of strategies.

We also provide a complete replication package that includes raw and cleaned datasets, search logs, screening and classification results, etc. The full resource is publicly available via Zenodo at: <https://doi.org/10.5281/zenodo.17799974>. The interactive tool and data set collectively enable researchers to explore intervention-outcome-KPI mappings, grounded in empirical evidence and traceable impact pathways.

¹⁰ Because many articles studied multiple Dev-X facets and KPIs simultaneously.

Table 4
Specific Dev-X interventions (146) employed that impacted software outcomes.

	Dev-X Intervention	Impact on Software Outcomes	KPIs
Cognitive Load	Controlling skin heat, heart rate, and breathing to decrease perceived difficulty [S116]	Improves code quality as measured by code metrics	Qual.
	Modifying brain signals to decrease cognitive load [S110], [S113]	Improves the correctness of code comprehension	Qual.
	Controlling difficulty through managing developer perceptions [S151]	Increases the defects rate they cause	Prod.
	Managing developer perceptions to reduce cognitive load [S23]	Improves tool usability based on the number of correct answers provided	Qual.
	Elevating code scattering that decreases developer flow [S102]	Predicts buggy classes	Qual.
	Elevating number of interruptions using interruption lag [S158]	Increases the project variations	Flow
	Employing eye-tracking, skin temperature, etc. to detect increased perceived difficulty [S118]	Elevates the number of interruptions using interruption lag metric	Flow
	Elevating developers' perceived coupling in code [S142]	Increases code coupling (structural, semantic, logical, and dynamic)	Qual.
	Adopting Cognitive-Driven Development in coding design [S126]	Reduces cognitive effort by limiting language constructs and intrinsic complexity	Qual.
	Heightening interruptions in developer perceptions [S19]	Increases the frequency of task switching and time spent on them	Prod.
	Activating specific brain areas to improve developer concentration [S114], [S153], [S7]	Decreases the difficulty of code comprehension	Qual.
	(De)Oxygenating haemoglobin values to increase cognitive load [S6]	Undermines code readability and increases code complexity	Qual.
	Leveraging eye tracking, brain signals, and skin temperature to increase cognitive load [S115], [S116], [S1], [S37]	Increases the challenge of performing tasks, Increases the difficulty of code comprehension	Prod., Qual., Prod.
	Expanding developers' perceived interruptions [S29]	Increases the number of modifications/edits	Prod., Flow
	Integrated LLM for code explanations/summarization [S81]	Improved code comprehension and debugging performance	Qual.
	Maintaining developer agency through trust, and satisfaction in automated bug repair [S120]	Reduces the manual bug-fixing workload	Prod.
	Applying memory tests using psychometric to improve developer attention [S47]	Improves correctness of test cases passed, readability of code, and time required to complete tasks	Qual., Prod.
	Using No-code builder for new developers [S87]	Improves the accuracy and correctness of the applications	Prod.
	Adopting targeted reading behavior to decrease mental/physical/temporal demand, and frustration [S57]	Hightens programmer efficacy	Prod., Proc. Eff., Qual.
	Desynchronizing the EEG bands for varied cognitive load [S111]	Explains the programming expertise for comprehension activities	Res. Util.
	Utilizing functional magnetic resonance imaging for varied complexity levels [S53]	Identifies code properties affecting cognition (textual size, vocabulary size)	Qual.
	Varying developer concentration based on brain areas activated [S154], [S155]	Explains the degree of concordance distinguishing the prose and code writing activities	Res. Util.
	Adopting test-driven development to improve enjoyment, challenge-skill balance, and sense of control [S82]	Induces flow state characterized by total immersion, focus, and involvement	Flow
	Aggravating the developers' perception for code smells [S125], [S34], [S143]	Improves performance of smell classification techniques into low, medium, high criticality	Qual.
	Employing Zero-shot LLMs for automated emotions recognition [S90]	Informs strategies to improve collaboration, reduce frustration, and enhance overall project outcomes	Dev. WB, Proj. Succ.
	Boosting sentiment scores on bug reports captured using SentiStrength-SE [S136], [S138]	Anticipates the code changes for bugs	Qual.
	Changing in levels of valence, arousal, dominance [S49]	Explains the design smells (technical debt) in code	Qual.
	Music listening to control positive/negative affects [S39]	Improves weekly quality of work	Prod.
	Cutting the negative affect score in security-related commits [S65]	Improves application security	Qual.
	Minimizing sentiments scores in commits [S78]	Adds to the modifications required in files	Qual.
	Not handling the perceived happiness during COVID-19 [S76]	Negatively affects the perceived work results and productivity	Prod.
	Guiding the types of terms developers' use during communication (developer attitude) [S10]	Shortens task completion time	Prod.
	Elevating developers' perceived negative feelings (e.g., frustration) [S137]	Increases the code review time and rounds	Qual.
	Elevating perception gap between developers [S139]	Increases requirement uncertainty	Qual.
	Controlling valence, arousal, dominance in bug reports to reduce burnouts [S44], [S4]	Identifies bug severity, shortens bug resolution time	Prod., Qual.
	Regulating affective values developers' perceives [S67]	Decreases task/relationship conflicts, Improves the success using requirement compliance	Proj. Succ., Team Dyn.
	Regulating affective values of comments captured using Sentistrength tool [S72]	Anticipates task types (e.g., bug-fixing, feature implementation, refactoring, etc.)	Proj. Succ.
	Regulating affective values of questions captured using Sentistrength tool [S68], [S71]	Increases the likelihood of question acceptance	Qual.

(continued on next page)

Table 4 (continued).

	Dev-X Intervention	Impact on Software Outcomes	KPIs
Emotional States	Enhancing developers' perceived emotions at work[S73]	Improves developer performance and process adherence using event proportion, sprint grade, etc.	Prod., Proc. Eff.
	Enhancing developers perceived happiness at work[S77]	Decreases developer isolation by feeling connected	Dev. WB
	Enhancing developers' perceived emotions at work[S89]	Improves developers perceived productivity	Prod.
	Boosting developers perceived happiness at work [S11], [S38], [S98]	Enhances creativity in accomplishing tasks, Improves code quality, productivity, or process adherence	Qual., Prod., Proc. Eff., Prod.
	Managing sentiments for commit messages captured using SentiStrength tool [S59]	Anticipates successful builds	Proj. Succ., Prod.
	Regulating valence score in mailing lists captured using SentiStrength tool [S62]	Classifies active/inactive developers	Flow
	Fostering positive emotions that developers' perceive at work [S94]	Improves performance in terms of number of commits	Prod.
	Restraining emotional intensity that developers' perceive at work [S24], [S27]	Increases disruptions at work, Reduces programming performance	Flow, Prod.
	Implementing non-invasive biometric sensors (EDA, HR via Empatica E4 wristband) for emotion capturing [S70]	Explains the perceived productivity of developers	Prod.
	Implementing empathy improving strategies like reducing session pace, asking follow-up questions, supporting struggling users, etc. [S91]	Enhances empathy between developers and end users	Team Dyn.
	Controlling arousal and valence as perceived [S54]	Improves debugging performance and number of tasks completed	Prod., Qual.
	Handling cognitive, affective, and behavioral empathy as developer perceives [S52]	Improves team learning about projects, Accelerates time to market	Prod., Proj. Succ.
	Improving satisfaction developer perceives [S13]	Improves project outcome (pass/fail)	Proj. Succ.
	Managing avg. sentiment in commit, and control (e.g., # bug priority) [S75]	Decreases bug resolution time	Prod., Qual.
	Regulating developers perceived empowerment [S134]	Decreases stress and improves team performance	Dev. WB, Prod.
	Controlling valence and arousal as perceived by developers [S28], [S2]	Improves developers perceived flow, Enhances developers perceived productivity	Flow, Prod.
	Boosting happiness, excitement, and control of developers as perceived [S150]	Enhances developers perceived competency	Res. Util.
	Adhering to XP methodology at work [S56]	Improves positive affects in developers	Dev. WB, Proc. Eff.
	Performing Retrospection to improve self-regulation [S22]	Provides better design recommendations	Qual.
	Controlling sentiment scores on commit messages captured using SentiStrength tool [S144]	Anticipates the types of activities developers like to perform	Res. Util.
	Controlling sentiments score for commits captured using SentiStrength tool [S64]	Anticipates the differences in programming languages, and the day of week commit made	Res. Util.
	Employing Agile to improve work engagement and resource depletion [S93]	Explains the impact of Agile ISD practices use	Developer Well-Being, Proc. Eff.
	Including of emotional feedback in agile retrospective meetings using EmoVizPhy tool [S43]	Reflects the causes of unhappiness	Team Dyn.
	Handling brain signals, eye blinking, and jaw clenching to improve valence, arousal, and dominance [S5]	Increases the difficulty of code comprehension	Prod., Qual.
	Managing emotional dissonance through developer perceptions [S26]	Aggravates developer's work exhaustion	Dev. WB
	Rising emotional responses for requirement changes [S40]	Increases technical debts and difficulty of code comprehension	Qual., Proj. Succ.
	Growing intensity of emotion scores in bugs [S80]	Elevates bug severity between normal, minor, or trivial classes	Qual.
	Not dealing with level of fatigue as developer perceives [S58]	Decreases computer interaction performance	Prod., Dev. WB
	Managing team/job fitness to reduce strain levels due to rapid changes [S18]	Increases the amount of workload a developer can handle	Dev. WB
	Increasing toxicity in GitHub commit messages [S122]	Improves code quality using duplicated lines density, security remediation effort	Qual.
	Controlling valence and arousal scores [S61]	Anticipates future changes in requirements	Qual.
	Lowering developers perceived stress [S134]	Decreases conflicts and ambiguity of roles	Team Dyn.
	Reducing morale as developer perceives at work place [S12], [S152]	Increases technical debts	Qual., Team Dyn.
	Minimizing social (pressure) work rate [S106]	Increases degree of multitasking (e.g., commits per project, switches)	Flow
	Regulating emotional intelligence through awareness of self and others' emotions [S14]	Improves trust among developers	Team Dyn.
	Suppressing joy, love, politeness, and elevated anger and sadness in developer perceptions [S147]	Heightens developer turnover risk	Res. Util.
	Monitoring emotions using tracking tools in requirement change handling [S41]	Improves emotional awareness and management during RC handling	Dev. WB, Prod.
	Watching high arousal movie clips [S3]	Improves debugging performance	Prod., Qual.

(continued on next page)

Table 4 (continued).

	Dev-X Intervention	Impact on Software Outcomes	KPIs
Motivation	Characterizing tasks with optimal challenge, high motivation, and no distractions [S84]	Improves the perceived productivity and project success	Flow
	Utilizing Developer Commit Ratio (DCR) to enhance developer experience [S86]	Tends to perform wider variety of refactoring operations	Qual., Prod.
	Enhancing intrinsic motivation through perceptions [S15]	Reduces development effort and enhances willingness to remain	Prod., Res. Util.
	Enhancing motivation through perceptions [S30]	Improves developers perceived performance	Prod.
	Performing early deployments that enhance motivation [S20]	Increases the frequency of deployments	Prod.
	Leveraging cognitive-walthrough on emails, blogs, etc. to enhance perception about testing infrastructure [S69]	Improves testers perception of the value of their contribution	Qual.
	Enhancing developers perceived productivity in work-sessions [S101]	Reduces work fragmentation that requires context switching	Flow, Prod.
	Regulating job enthusiasm through new ideas, and feedbacks [S105]	Improves developers perceived productivity	Prod.
	Elevating impact of agile methodologies through developer perceptions [S31]	Improves developers perceived productivity	Prod., Proc. Eff.
	Improving the perceived productivity among developers [S25]	Increases the number of commits	Prod.
	Handling work satisfaction and burnout in developer comments [S95]	Improves the organizational culture and sense of belonging	Team Dyn.
	Enhancing engagement with computers (mouse, keyboard use) [S21]	Decreases number of interruptions	Flow
	Improving the perceived motivation of developers [S8]	Improves project outcome (pass/fail)	Proj. Succ.
	Setting reflective achievement goals from developers [S107]	Refines the development work habits	Res. Util.
Others	Addressing developer information needs for bug report [S17]	Improves quality of bug report	Qual.
	Augmenting relevant data in emotion recognition [S45]	Improves emotion classification tool performance	Qual.
	Employing emotion recognition for social and human factors [S79]	Improves the perceived productivity of developers	Prod.
	Enhancing episodic developer experience [S83]	Increases the productivity levels of developers using LCDPs	Prod.
	Employing quality code summaries in development [S141]	Improves the correctness of code	Qual.
	Boosting conscientiousness personality traits [S9]	Increases depth-first coding	Qual.
	Enhancing developers' quality employing perceptions [S33]	Increases fault-proneness of files	Qual.
	Improving developers' perceived well-being (e.g., stress) [S85]	Minimizes inefficiencies in software development	Prod., Proj. Succ.
	Employing WHO standard for developers well-being [S133]	Improves developers perceived productivity	Dev. WB, Prod.
	Incorporating challenge in the task as developer perceives [S51]	Improves the value of a software feature	Qual.
	Improving the extent of social presence and politeness in sourceforge messages [S50]	Enhances the sustainability of the project	Proj. Succ.
	Integrating ethics into daily practice of AI developers [S66]	Streamlines ethical considerations in development processes	Proc. Eff.
	Activating policies addressing autonomy and relatedness [S128]	Stables productivity levels and increase satisfaction	Prod., Developer Wellbeing
	Addressing role-specific overhead communication and extra work [S117]	Enhances development process effectiveness and team communication	Proc. Eff., Team Dyn.
	Adopting agile methodology [S55]	Reduces anxiety, depression and improves contentment, enthusiasm	Dev. WB, Proc. Eff.
	Adopting HRM practices like: developmental appraisal, equitable reward, and training [S88]	Increases the likelihood of successful completion of projects	Proj. Succ.
	Aligning process with developers perceived values [S160]	Improves the impact of Lean and Agile methods	Proc. Eff.
	Assigning undesirable tasks among all team members [S74]	Increased developer stress and frustration and decreased morale	Team Dyn., Dev. WB
	Employing interpersonal conflicts resolution methods, collaborative problem-solving, and conflict management strategies [S96]	Improves review code quality and team collaboration	Qual., Team Dyn.
	Rising honesty violations in mobile app reviews [S60]	Degrades the developers' experiences with app quality	Qual.
	Employing ChatGPT to generate human-value user stories during requirements elicitation [S63]	Increases internal completeness and clarity of user stories	Proj. Succ.
	Employing strategies for managing emotions, relationships, etc. [S99]	Enhances goal-setting, collaboration among team members	Prod., Team Dyn.
	Leveraging dashboards and tools for developer awareness of tasks and colleagues performing [S42]	Strengthens developer trust in team	Team Dyn.
	Improving developer perceptions about their role (core/peripheral) [S145]	Improves developer contributions and centrality	Prod., Res. Util.
	Assigning politeness score to comments [S48]	Predicts team national diversity using Blaus Diversity Index	Team Dyn.
	Adopting individualistic/collectivistic cultures awareness strategies [S100]	Enhances team collaboration and cohesion, knowledge sharing, and retention	Proc. Eff., Team Dyn.
	Utilizing the direct and transitive collaboration histories for developers compatibility [S129]	Improves project alignment by recommending compatible team members	Proj. Succ.
	Enhancing developers openness as they perceived at work [S132]	Improves pair programming as academic performance	Proc. Eff., Prod.

(continued on next page)

Table 4 (continued).

	Dev-X Intervention	Impact on Software Outcomes	KPIs
Values	Enhancing code value through developer perceptions [S149]	Enhances the perceived impact of development	Qual.
	Growing the ownership perception about some code [S36]	Improves developer commitment based on number of asserts	Qual., Res. Util.
	Adopting knowledge sharing between developers to foster mutual trust [S121]	Improves developers commitment and mutual trust	Team Dyn.
	Varying interpersonal trust dimensions in OSS teams [S46]	Explains the OSS sustainability	Proj. Succ.
	Using the generative AI to assess system/output quality, functional value, and goal maintenance [S112]	Enhances trust and behavioral intentions towards genAI tools	Qual., Team Dyn.
	Adopting management practices: fair rewards, and collaborative team structures [S108]	Improves developers perceived productivity	Prod.
	Guiding through explicit checklist-based review strategies [S104]	Improves defect detection effectiveness and reduces the cognitive strain	Qual., Dev. WB
	Leveraging betweenness, homophily (congruence) between developers [S16]	Reduces request resolution time	Prod., Proc. Eff.
	Employing eye-tracking while performing tasks [S103]	Differentiates developers based on workloads	Quality Outcomes, Team Dyn.
	Regulating developers job/team fit through response time, contributions, etc. [S124]	Enhances developer retention as based on days of activeness	Team Dyn.
	Varying the level of perceived commitment [S130]	Explains developer-organization value incongruence	Team Dyn.
	Employing model-driven engineering for modeler experience [S92]	Results in successful infrastructure-as-code, Low-code, MBSE, etc.	Prod., Qual., Proc. Eff.
	Employing developer perceptions to assess code quality metrics [S140]	Increases the metrics relevance for refactoring	Qual.
	Meeting the developers values at work place [S135]	Improves likelihood of OSS contributions	Proj. Succ.
	Employing developers closeness and betweenness values to capture social presence [S123]	Decreases the test or post-release failures rate of files	Proj. Succ., Qual.
	Augmenting more diversity dimensions (age, sex, ethnicity, education, social background, and work experience) [S127]	Improve team stress management, and collaboration skills	Team Dyn.
	Using less restrictive license types of project [S131]	Improves developers interests based on number of contributions	Proj. Succ.
	Not controlling the human values violations in stackoverflow posts [S35]	Downvotes the stackoverflow posts	Team Dyn.
	Improving developer perception about the impact of their work More impactful work and autonomy [S108]	Improves developers perceived satisfaction	Team Dyn.
	Adopting agile methodologies for communication improvements, and cultural models [S97]	Mitigates stress factors like cultural and temporal distances	Dev. WB, Team Dyn.
	Utilizing retrospection and automation tools to promote awareness and workload management [S109]	Mitigates fatigue and turnover intentions	Dev. WB, Team Dyn.
	Leveraging the number of comments exchanged between developers to prioritize them [S32]	Improves bug triaging accuracy	Qual.
	Promoting gender diversity through hiring, cultural codes, and gender-neutral workplace [S119]	Enhances idea sharing, decision-making, and innovation in teams	Team Dyn.
	Varying the developer perceptions about the impact of social network metrics [S148]	Explains the actual effectiveness of the metrics	Qual.
	Varying the developer perceptions about the impact of testing techniques [S146]	Explains the effectiveness of the code testing techniques	Qual.
	Varying the developer perceptions about the impact of design patterns [S156]	Explains the impact of code quality attributes e.g., modularity	Qual.
	Varying the developer perceptions about the impact of programming languages [S157]	Distinguishes the good/bad linguistic patterns	Res. Util.
	Varying the developer perceptions about the impact of (peripheral/core) [S159]	Explains the feedback interval and acceptance for review	Res. Util.

5. Discussion

This section synthesizes our findings across RQ1 and RQ2, contextualizes longitudinal trends, identifies gaps in Dev-X research, and outlines contributions to practice and future work.

5.1. RQ1 and RQ2 findings: Practical and theoretical implications

Our analysis of 160 intervention-focused empirical studies reveals important insights for both practitioners and researchers.

Practical Implications.

- **Quality as a Central Outcome:** Contrary to prevailing assumptions, *quality outcomes* (e.g., code readability, defect rates) were more frequently impacted by Dev-X interventions than *productivity* ($\langle Q \rangle$ vs. $\langle P \rangle$ links; see Fig. 6). Organizations should thus broaden Dev-X strategies beyond throughput measures to encompass software engineering quality dimensions.

- **Intervention Traceability & On-demand Filtering:** Only 39 studies explicitly report full *Intervention* $\rightarrow$ *Outcome* $\rightarrow$ *KPI* chains. To make these instantly usable, our interactive Ready-Reckoner¹¹ provides free-text search (Title/Intervention/Outcomes) and facet filters (Facet, Outcome, Business KPI, Relationship Type), returning card-level tuples with study IDs. This lets practitioners shortlist evidence-backed interventions at the point of need.
- **Strategic Contextualization:** Different intervention types aligned with distinct KPIs: cognitive load reducers predominantly supported quality and efficiency; values-aligned practices influenced retention and project success. Practitioners must select interventions aligned with their strategic goals rather than generic productivity improvements [2,3].
- **Instrument underrepresented KPIs before scaling:** *Flow*, *Process Efficiency*, and *Resource Utilization* are comparatively sparse

¹¹ <https://ready-reckoner-devx.netlify.app/>

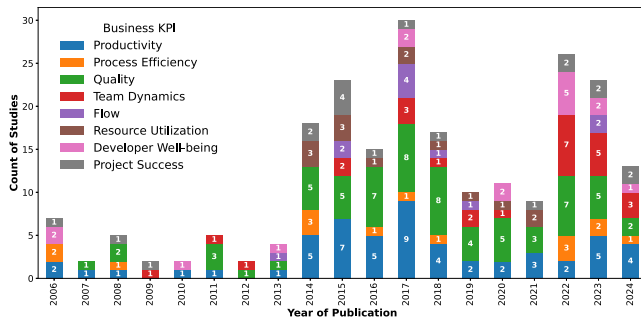


Fig. 7. Publications per Year by Business KPI Impacted (2006–2024)

($\langle Flow \rangle$, $\langle Proc \rangle$, $\langle Res \rangle$ links). Add lightweight instrumentation (e.g., task-switch counts, cycle time, utilization indices) and run small pilots before wider rollout.

- **Match intervention families to KPIs:** In our mappings, cognitive-load reductions more often co-occur with *Quality/Efficiency*, while values/HRM-oriented practices more often co-occur with *Team Dynamics* and *Project Success* (Fig. 6, Table 2). Use this mapping to select interventions that fit declared KPIs rather than generic “productivity” bets.

Theoretical Implications.

- **Shift Towards Affective and Social Constructs:** Emotional states [3] and values were more frequently studied facets than traditional cognitive constructs [1]. Future theories should incorporate affective and socio-organizational dimensions more centrally.
- **Methodological Gaps (causal and longitudinal):** Overreliance on correlational studies constrains theoretical advancement. Only 21 relationships are reported as cause-effect and only 39 studies report full *Intervention*→*Outcome*→*KPI* chains; longitudinal designs remain limited. Stronger causal modeling and longitudinal evaluation are needed [6].
- **Complex Multi-Channel Interventions:** Our mappings show multi-facet effects in a non-trivial subset of studies (see co-occurrence in our ready-reckoner <https://ready-reckoner-devx.netlify.app/> and counts in Fig. 6). In keeping with our taxonomy (as presented in Fig. 4), we refer to facets explicitly – Cognitive Load, Emotional States, Values, Motivation, and Other – rather than a generic “social” domain (team-level effects appear under Team Dynamics and Culture). Theoretical models should therefore allow compound pathways across these defined facets.
- **Standardize reporting and traceability:** Adopt the $\{Intervention \rightarrow Outcome \rightarrow KPI\}$ schema and release study-ID maps/codebooks (replication package) to strengthen reproducibility and auditability (cf. [23]).
- **Broaden KPI coverage with validated measures:** Under-represented KPIs (Flow, Process Efficiency, Resource Utilization) need shared, validated operationalizations to avoid search/measurement bias; include explicit KPI definitions, instruments, and raw variables in artifacts.

5.2. Recognized interventions and software outcomes: Practical mappings

Our analysis of 146 unique interventions demonstrates that Dev-X manipulations influence a diverse set of external software outcomes, offering leverage points beyond productivity enhancement.

Diversity of Outcomes Influenced. Interventions were found to impact:

- **Cognitive Outcomes**—e.g., improved code comprehension and debugging accuracy (via biometric feedback [S116, S110]).
- **Social Dynamics**—e.g., trust-building and enhanced collaboration (via retrospection feedback [S43]).
- **Artifact Quality**—e.g., enhanced readability and maintainability (via Large Language Model (LLM) summarization [S112]).
- **Developer Engagement**—e.g., flow state induction and reduced emotional exhaustion (via workload dashboards [S109]).

Mapping Characteristics. Interventions varied along two key dimensions:

1. **Multi-Outcome Influence:** For example, Emotional States-targeting interventions show co-occurrences with all (specially with Quality, Team Dynamics, and Productivity), see dense cells in Fig. 6; representative exemplars include [S11, S73, S75]
2. **Specificity:** In contrary to the Emotion States, Cognitive Load interventions concentrate on KPIs mostly related to the code comprehension/attention with comparatively fewer links to Team Dynamics and Culture and overall Project Success (Fig. 6); e.g., [S110, S114, S153]

5.3. Longitudinal shifts in Dev-X intervention design and KPI focus

Using the year-by-KPI series (Fig. 7) – where each segment denotes the count of KPI tags assigned to studies in that year (multi-label; a study may map to multiple KPIs) – we observe a clear evolution in both intervention design and targeted KPIs. Three temporal phases are identified which are presented in Table 5.

Temporal Phases.

- **2006–2015 (Foundation):** Early emphasis on individual states and fundamentals – affect/mood [S3], motivation [S30], and initial cognitive-load studies – with KPI attention concentrated on *Productivity* (27.1%) and *Quality* (25.7%), while *Project Success* also appears non-trivially (12.9%). In contrast, *Flow* is present but sparse in this period (4.3%).
- **2016–2020 (Consolidation towards Quality & Productivity):** Broader use of sentiment/VAD [S136] [S148] [S64] [S65] [24] and cognitive-load measures coincides with *Quality* becoming the dominant KPI focus (38.6%), rising by +12.8 percentage points compared to 2006–2015 (25.7%). *Productivity* remains stable (26.5%), with modest growth in *Flow* (7.2%). Pandemic-era work (2020) foregrounds well-being [S85] [S93] [S117], contributing half of the period’s well-being tags (2 of 4).
- **2021–2024 (Strategic Integration with AI & Values):** A marked rise in organizational and sociotechnical KPIs is visible, with *Team Dynamics* increasing to 21.1% (up from 8.4% in 2016–2020) and *Developer Well-being* to 11.3% (up from 4.8%). *Project Success* shows sustained attention (9.9%). In parallel, intervention designs increasingly include *AI-assisted* [S148] [S42] and *no/low-code* and GenAI-specific constructs [S112] (trust, zero-shot emotion analysis).

Trend Summary. The field shifts from individual performance enhancement towards organizationally aligned, sociotechnical Dev-X interventions, with a late acceleration (2021–2024) in AI-adjacent and values/ethics-oriented work. Persistent challenges remain in causal validation and traceable, end-to-end links from intervention to business outcome.

5.4. Cross-facet and longitudinal gaps in Dev-X research

Despite the progress observed, several empirical gaps remain:

Cross-Facet Interactions. Few studies simultaneously analyzed how multiple Dev-X facets (e.g., cognitive load × emotional states) interact

Table 5

KPI distribution across temporal phases (counts; share within phase). Totals (n) are KPI-tag counts, not unique papers.

Business KPI	2006–2015 ($n = 70$)	2016–2020 ($n = 83$)	2021–2024 ($n = 71$)
Productivity	19 (27.1%)	22 (26.5%)	14 (19.7%)
Process Efficiency	6 (8.6%)	3 (3.6%)	6 (8.5%)
Quality	18 (25.7%)	32 (38.6%)	17 (23.9%)
Team Dynamics	5 (7.1%)	7 (8.4%)	15 (21.1%)
Flow	3 (4.3%)	6 (7.2%)	2 (2.8%)
Resource Utilization	6 (8.6%)	6 (7.2%)	2 (2.8%)
Developer Well-being	4 (5.7%)	4 (4.8%)	8 (11.3%)
Project Success	9 (12.9%)	3 (3.6%)	7 (9.9%)

to affect outcomes [S4] [S52] [S64] [S67] [S108] [S134]. This remains a major theoretical and practical blind spot.

Longitudinal Validation. Only a small subset of studies employed longitudinal designs [S77] [S85] [S131] to validate sustained impacts of interventions. Most evidence remains cross-sectional, limiting causal inference.

Future work should prioritize multifaceted, longitudinal intervention evaluations to better reflect real-world software engineering environments.

5.5. Traceability and causality limitations

Although many studies showed positive correlations between Dev-X interventions and outcomes, only 21 established **causal relationships**. Furthermore, only 39 studies presented a full directional chain to business KPIs.

Thus, practitioners adopting interventions must remain cautious about overgeneralizing findings without considering context, validation strength, and intended KPI impact.

5.6. Interactive ready-reckoner: What it does and how it helps

Data model (aligned with RQ1/RQ2). The tool is a read-only, searchable view over the tuples extracted in this study:

{Facet, Intervention, Software Outcome(s),
Business KPI(s), Relationship Type, Study ID}.

Each on-screen card corresponds to a row in our released mapping tables (replication package), ensuring study-ID traceability.

Core functionality. Users can (i) *search* by Title/Intervention/Outcomes, (ii) *filter* by Facet, Outcome, Business KPI, and Relationship Type (correlation, cause-effect, comparison, prediction/classification, qualitative association), and (iii) *inspect* the evidence summary per study. The index mirrors the full set analyzed in this SMS.

Worked examples. (1) *Quality-first triage:* Filter Business KPI = Quality Outcomes and optionally Relationship = Cause--Effect/Prediction to shortlist interventions like load-reduction or summarization aids that improve comprehension/defect outcomes. (2) *Flow/interruptions:* Filter Business KPI = Flow to surface interruption-reduction and engagement interventions with their observed outcomes (e.g., fewer task switches, perceived flow).

Practical value and limits. The Ready-Reckoner reduces the effort of translating the mapping into action—moving from *goal/KPI* → *filters* → *shortlist*. It does not impute causal strength beyond what is reported; instead, it exposes the study design via the Relationship Type and links back (by ID) to our tables for deeper inspection.

6. Threats to validity

Following the secondary-study framing of Ampatzoglou et al. [23], we group threats into *Study Selection Validity*, *Data Validity*, and *Research Validity*, each with mitigations and residual risk.

Study Selection Validity (search & screening). *Query construction & DB coverage.* Alternate terminology may escape our string. *Mitigations:* high-frequency uni/bi-grams from seed studies, pilot refinements, syntax adaptation per library, and backward snowballing. *Residual:* edge terms may still be missed. *Screening bias.* Single-reviewer inclusion errors. *Mitigations:* double-blind screening with predefined criteria and conflict resolution. *Residual:* low, given adjudication.

Data Validity (extraction, coding, publication bias). *Extraction/coding subjectivity.* Assigning facets, relationship types, and KPIs involves judgment. *Mitigations:* controlled vocabulary + extraction handbook, pilot on 16 papers, inter-rater check (Fleiss' $\kappa = 0.79$), consensus meetings. *Residual:* some subjectivity remains; codebooks/data released for audit. *Classification robustness.* Schema errors could bias mappings. *Mitigations:* axial coding with documented definitions, iterative consolidation, and (where applicable) open card sorting with agreement thresholds. *Residual:* taxonomy may evolve with new evidence. *Publication bias.* Positive/venue preferences. *Mitigations:* multi-library search, Google Scholar, venue diversity, snowballing; full dataset public. *Residual:* grey literature not systematically included. *Implicit KPI inference.* Some KPIs inferred from software outcomes. *Mitigations:* explicit tagging and study-ID traceability in released tables/tool. *Residual:* limited interpretive bias possible.

Research Validity (generalization & conclusions). *Generalizability.* Although 160 studies (2006–2024) span domains, results may not transfer to all contexts (e.g., fast-moving AI-assisted workflows). *Mitigations:* report counts by facet/KPI; avoid context-free prescriptions; tool enables context-aware filtering. *Conclusion strength.* Many primary studies are correlational; only a subset provide full {Intervention→Outcome→KPI} chains. *Mitigations:* we separate correlational vs. causal links, report full-chain counts elsewhere, and avoid unwarranted causal claims.

Repeatability & Transparency. We document protocol/criteria, record screening/extraction decisions, and release dataset, schema, and tool to support replication and independent verification.

7. Conclusion and future work

This systematic mapping study provides a comprehensive synthesis of Developer Experience (Dev-X) interventions and their pathways to software outcomes and business KPIs. By analyzing 160 studies across 18 years, we identify a maturing research trajectory: from early emphasis on individual cognitive load, through affective and social dynamics, to strategic integration of values-aligned interventions targeting organizational resilience.

Our findings nuances the common emphasis on productivity by showing quality outcomes are at least as prominent in the mapped evidence. Instead, quality-related KPIs – such as code correctness, maintainability, and collaboration effectiveness – emerged as the most consistently influenced outcomes. Furthermore, although emotional states and value alignment dominate the Dev-X facets explored, our longitudinal analysis highlights an increasing attention towards holistic well-being, project success, and retention as core strategic goals.

Table A.6

Dataset: set of articles included in review.

ID	Title	ID	Title
S1	Using psycho-physiological measures to assess task difficulty in software development	S81	Experiences from Using Code Explanations Generated by Large Language Models in a Web Software Development E-Book
S2	Stuck and frustrated or in flow and happy: Sensing developers' emotions and progress	S82	Test-Driven Development Benefits Beyond Design Quality: Flow State and Developer Experience
S3	Mood independent programming	S83	Measuring End-user Developers' Episodic Experience of a Low-code Development Platform
S4	Mining valence, arousal, and dominance: possibilities for detecting burnout and productivity?	S84	Flow Experience in Software Engineering
S5	Mining biometric data to predict programmer expertise and task difficulty	S85	Daily questionnaire to assess self-reported well-being during a software development project
S6	The Effect of Poor Source Code Lexicon and Readability on Developers' Cognitive Load	S86	Behind the scenes: On the relationship between developer experience and refactoring
S7	A look into programmers' heads	S87	Low-code vs. the developer: An empirical study on the developer experience and efficiency of a no-code platform
S8	Factors that motivate software engineering teams: A four country empirical study	S88	Influence of HRM practices on innovation in software engineering: the mediating role of developer experience
S9	Links between the personalities, styles and performance in computer prog.	S89	Emotions in the software development process
S10	Exploring the links between software development task type, team attitudes and task completion performance: Insights from the Jazz repository	S90	Uncovering the Causes of Emotions in Software Developer Communication Using Zero-shot LLMs
S11	What happens when software developers are (un)happy	S91	Enablers and Barriers of Empathy in Software Developer and User Interactions: A Mixed Methods Case Study
S12	The influence of Technical Debt on software developer morale	S92	Systematizing modeler experience (MX) in model-driven engineering success stories
S13	Software developer perceptions about software project failure: a case study	S93	Sprint Zeal or Sprint Fatigue? The Benefits and Burdens of Agile ISD Practices Use for Developer Well-Being
S14	Emotional intelligence: The key to mitigating stress and fostering trust among software developers working on information system projects	S94	Importance of positive emotions in software developers' performance: a narrative review
S15	Understanding Informal Control Modes on Software Platforms—The Mediating Role of Third-Party Developers' Intrinsic Motivation	S95	A Model for Understanding and Reducing Developer Burnout
S16	Socio-technical congruence: a framework for assessing the impact of technical and work dependencies on software development productivity	S96	Interpersonal Conflicts During Code Review
S17	What makes a good bug report?	S97	Stress, motivation, and performance in global software engineering
S18	Performance and Strain Levels of IT Workers Engaged in Rapidly Changing Environments: A Person-Job Fit Perspective	S98	Consequences of Unhappiness While Developing Software
S19	Software Developers' Perceptions of Productivity	S99	Supporting Emotional Intelligence, Productivity and Team Goals while Handling Software Requirements Changes
S20	Continuous Deployment at Facebook and OANDA	S100	Impact of individualism and collectivism cultural profiles on the behavior of software developers: A study of stack overflow
S21	Reducing interruptions at work: A large-scale field study of flowlight	S101	The Work Life of Developers: Activities, Switches and Perceived Productivity
S22	Design Recommendations for Self-Monitoring in the Workplace: Studies in Software Development	S102	A Developer Centered Bug Prediction Model
S23	Improving the Developer Experience with a Low-Code Process Modelling Language	S103	Studying Developer Eye Movements to Measure Cognitive Workload and Visual Effort for Expertise Assessment
S24	Applicability of Emotion Recognition and Induction Methods to Study the Behavior of Programmers	S104	Doexplicit review strategies improve code review performance? Towards understanding the role of cognitive load
S25	Code and Commit Metrics of Developer Productivity: A Study on Team Leaders Perceptions	S105	What Predicts Software Developers' Productivity?
S26	How agile software development methods reduce work exhaustion: Insights on role perceptions and organizational skills	S106	Socio-Technical Work-Rate Increase Associates With Changes in Work Patterns in Online Projects
S27	How do you feel, developer? An explanatory theory of the impact of affects on programming performance	S107	Enabling Good Work Habits in Software Developers through Reflective Goal-Setting
S28	Do feelings matter? On the correlation of affects and the self-assessed productivity in software engineering	S108	Towards a Theory of Software Developer Job Satisfaction and Perceived Productivity
S29	Work fragmentation in developer interaction data	S109	Too Drained from Being Agile? The Self-Regulatory Effects of Agile ISD Practices Use and their Consequences for Turnover Intention
S30	Software Developer Motivation in a High Maturity Company: a Case Study	S110	Assessing the contribution of the individual alpha frequency (IAF) in an EEG-based study of program comprehension
S31	The perceived impact of the agile development and project management method scrum on information systems and software development productivity	S111	Understanding Programming Expertise: An Empirical Study of Phasic Brain Wave Changes
S32	Developer Prioritization in Bug Repositories	S112	What Guides Our Choices? Modeling Developers' Trust and Behavioral Intentions Towards GenAI
S33	The influence of developer quality on software fault-proneness prediction	S113	Towards an affordable brain computer interface for the assessment of programmers' mental workload
S34	Do they Really Smell Bad? A Study on Developers' Perception of Bad Code Smells	S114	Measuring neural efficiency of program comprehension
S35	Human Values Violations in Stack Overflow: An Exploratory Study	S115	Measuring software developers' perceived difficulty with biometric sensors
S36	Assert Use in GitHub Projects	S116	Using (bio)metrics to predict code quality online
S37	Psycho-physiological measures for assessing cognitive load	S117	Requirements Quality vs. Process and Stakeholders' Well-Being: A Case of a Nordic Bank
S38	Happy software developers solve problems better: psychological measurements in empirical software engineering	S118	Interruptibility of Software Developers and its Prediction Using Psycho-Physiological Sensors
S39	The effect of music listening on work performance	S119	Benefits and Difficulties of Gender Diversity on Software Development Teams: A Qualitative Study

(continued on next page)

Table A.6 (continued).

ID	Title	ID	Title
S40	The Emotional Roller Coaster of Responding to Requirements Changes in Software Engineering	S120	How do Developers Really Feel About Bug Fixing? Directions for Automatic Program Repair
S41	A Framework for Emotion-Oriented Requirements Change Handling in Agile Software Engineering	S121	Antecedents of coordination effectiveness of software developer dyads from interacting teams: an empirical investigation
S42	Bridging gap between awareness and trust in globally distributed teams	S122	Exploring the Impact of Toxic Comments in Code Quality
S43	Towards Supporting Emotion Awareness in Retrospective Meetings	S123	Predicting Failures with Developer Networks and Social Network Analysis
S44	Bootstrapping a lexicon for emotional arousal in software engineering	S124	Who Will Remain? - An Evaluation of Actual Person-Job and Person-Team Fit to Predict Developer Retention in FLOSS Projects
S45	Data Augmentation for Improving Emotion Recognition in Software Engineering Communication	S125	Developer-Driven Code Smell Prioritization
S46	Interpersonal Trust in OSS: Exploring Dimensions of Trust in GitHub Pull Requests	S126	To What Extent Cognitive-Driven Development Improves Code Readability?
S47	Do Attention and Memory Explain the Performance of Software Developers?	S127	Exposing Software Engineering Students to Stressful Projects: Does Diversity Matter?
S48	How diverse is your team? Investigating gender and nationality diversity in GitHub teams	S128	Satisfaction and performance of software developers during enforced work from home in the COVID-19 pandemic
S49	Measuring affective states from technical debt	S129	Recommending People in Developers' Collaboration Network
S50	Understanding group maintenance behavior in Free/Libre Open-Source Software projects: The case of Fire and Gaim	S130	When Cultures Clash: Participation in Open Source Communities and Its Implications For Organizational Commitment
S51	Explaining over-requirement in software development projects: An experimental investigation of behavioral effects	S131	Determinants of open source software project success: A longitudinal study
S52	Antecedents and consequences of collective empathy in software development project teams	S132	Investigating the effects of personality traits on pair programming in a higher education setting through a family of experiments
S53	Program Comprehension and Code Complexity Metrics: An fMRI Study	S133	Pandemic programming
S54	Do moods affect programmers' debug performance?	S134	Technical systems development risk factors: The role of empowering leadership in lowering developers' stress
S55	The Impact of Agile Methodology on Well Being of Development Teams	S135	For Love or Money? Examining Reasons OSS Developers' Contributions
S56	The Positive Affect of the XP Methodology	S136	Understanding the Effect of Developer Sentiment on Fix-Inducing Changes: An Exploratory Study on GitHub Pull Requests
S57	The human experience of comprehending source code in virtual reality	S137	Predicting Developers' Negative Feelings about Code Review
S58	Characterizing and Predicting Mental Fatigue during Programming Tasks	S138	Is Developer Sentiment Related to Software Bugs: An Exploratory Study on GitHub Commits
S59	Sentiment Analysis of Travis CI Builds	S139	Direct and mediating influences of user-developer perception gaps in requirements understanding on user participation
S60	Automated detection, categorization and developers' experience with the violations of honesty in mobile apps	S140	Do design metrics capture developers perception of quality? an empirical study on self-affirmed refactoring activities
S61	Using the Affect Grid to Measure Emotions Lumbieras in Software Requirements Engineering	S141	A Human Study of Comprehension and Code Summarization
S62	The Role of Emotions in Contributors Activity: A Case Study on the GENTOO Community	S142	An Empirical Study on the Developers' Perception of Software Coupling
S63	Using ChatGPT to Generate Human-Value User Stories as Inspirational Triggers	S143	Smells are sensitive to developers! On the efficiency of (un)guided customized detection
S64	Sentiment analysis of commit comments in GitHub: an empirical study	S144	How Do Code Refactoring Activities Impact Software Developers' Sentiments? - An Empirical Investigation into GitHub Commits
S65	Security and emotion: sentiment analysis of security discussions on GitHub	S145	Classifying Developers into Core and Peripheral: An Empirical Study on Count and Network Metrics
S66	The ethical wisdom of AI developers	S146	On (Mis)perceptions of testing effectiveness: an empirical study
S67	Media, Mood, and Meetings: Related to Project Success?	S147	Predicting Community Smells' Occurrence on Developers by Sentiments
S68	Mining Successful Answers in Stack Overflow	S148	Socio-Technical Developer Networks: Should We Trust Our Measurements?
S69	How do Testers Feel it ? An Experience Report on Evaluating TX in an Industrial Context	S149	Towards Quantifying the Development Value of Code Contributions
S70	Emotions and Perceived Productivity of Software Developers at the Workplace	S150	Effects of Automated Competency Evaluation on Software Engineers' Emotions and Motivation: A Case Study
S71	Understanding question quality through affective aspect in Q&A site	S151	Evaluating the work of experienced and inexperienced developers considering work difficulty in software development
S72	Towards understanding and exploiting developers' emotional variations in software engineering	S152	Looking for Peace of Mind? Manage Your (Technical) Debt: An Exploratory Field Study
S73	Outcomes of Emotional Content from Agile Team Forum Posts	S153	Understanding Source Code with Functional Magnetic Resonance Imaging
S74	Like, dislike, or just do it? How developers approach software development tasks	S154	Neurological Divide: An fMRI Study of Prose and Code Writing
S75	Are Bullies More Productive? Empirical Study of Effectiveness vs. Issue Fixing Time	S155	Towards an Objective Measure of Developers' Cognitive Activities
S76	Happier and further by going together: The importance of software team behavior during the COVID-19 pandemic	S156	Behind the scenes: Developers' perception of multi-language practices
S77	Fixing the 'Out of sight out of mind' problem one year of mood-based microblogging in a distributed software team	S157	Linguistic antipatterns: what they are and how developers perceive them
S78	Analyzing developer sentiment in commit logs	S158	Task Interruptions in Requirements Engineering: Reality Versus Perceptions!
S79	An Instrument for Measuring Perception about Social and Human Factors that Influence Software Development Productivity	S159	Impact of developer reputation on code review outcomes in OSS projects: an empirical investigation
S80	Analyzing emotion words to predict severity of software bugs: a case study of open source projects	S160	Examining the Structure of Lean and Agile Values among Software Developers

Despite this evolution, critical gaps persist. Only a minority of studies established end-to-end causal pathways from intervention to software outcome to business KPI. Intervention focus remains concentrated around a narrow subset of outcomes, risking the neglect of critical but underexplored dimensions such as developer flow, social presence, and ethical decision-making. The fragmentation of empirical evidence underscores the urgent need for deeper validation and broader modeling of Dev-X phenomena.

To support practical adoption and facilitate future research, we developed an open-access Dev-X Ready-Reckoner tool. By providing traceable mappings between interventions, Dev-X facets, external software outcomes, and business KPIs, the tool enables practitioners to design more targeted, evidence-informed strategies aligned with organizational priorities.

Future Work. Several important directions emerge from this study. First, longitudinal and experimental research is needed to validate the sustained, causal impact of Dev-X interventions beyond cross-sectional snapshots. Second, future studies should explicitly model multi-channel effects, as many interventions influence cognitive, emotional, and social dimensions simultaneously. Third, expanding empirical focus towards underexplored outcomes – such as resource utilization, ethical decision-making, and organizational adaptability – would strengthen the strategic scope of Dev-X research. Finally, the rising use of AI-driven tools and emotion-aware environments introduces both opportunities and risks that merit focused investigation. Overall, advancing Dev-X scholarship will require broader, integrative, and causally grounded research frameworks that align with real-world organizational needs.

CRediT authorship contribution statement

Abdul Qayum: Software, Investigation, Formal analysis, Data curation. **Ayyaz Qureshi:** Formal analysis, Data curation. **Abdul Razzaq:** Conceptualization, Funding acquisition, Methodology, Supervision, Writing – review & editing.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Reports a relationship with that includes: Has patent pending to. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix

See Table A.6.

Data availability

Most of the Data is Already Available. More can be made available on request.

References

- [1] F. Fagerholm, J. Münch, Developer experience: Concept and definition, in: 2012 International Conference on Software and System Process, ICSSP, IEEE, 2012, pp. 73–77.
- [2] M.-A. Storey, T. Zimmermann, C. Bird, J. Czerwonka, B. Murphy, E. Kalliamvakou, Towards a theory of software developer job satisfaction and perceived productivity, *IEEE Trans. Softw. Eng.* 47 (10) (2019) 2125–2142.
- [3] A. Razzaq, J. Buckley, Q. Lai, T. Yu, G. Botterweck, A systematic literature review on the influence of enhanced developer experience on developers' productivity: Factors, practices, and recommendations, *ACM Comput. Surv.* 57 (1) (2024) 1–46.
- [4] J. Morales, C. Rusu, F. Botella, D. Quiñones, Programmer experience: A systematic literature review, *IEEE Access* 7 (2019) 71079–71094.
- [5] A. Nylund, A Multivocal Literature Review on Developer Experience, (Master's thesis), Aalto University, 2020.
- [6] M. Greiler, M.-A. Storey, A. Noda, An actionable framework for understanding and improving developer experience, *IEEE Trans. Softw. Eng.* 49 (4) (2022) 1411–1425.
- [7] L. Gonçalves, K. Farias, B. da Silva, J. Fessler, Measuring the cognitive load of software developers: A systematic mapping study, in: 2019 IEEE/ACM 27th International Conference on Program Comprehension, ICPC, IEEE, 2019, pp. 42–52.
- [8] W. Hussain, H. Perera, J. Whittle, A. Nurwidyanoro, R. Hoda, R.A. Shams, G. Oliver, Human values in software engineering: Contrasting case studies of practice, *IEEE Trans. Softw. Eng.* (2020).
- [9] K. Petersen, R. Feldt, S. Mujtaba, M. Mattsson, Systematic mapping studies in software engineering, in: 12th International Conference on Evaluation and Assessment in Software Engineering, EASE, BCS Learning & Development, 2008.
- [10] B. Kitchenham, O.P. Brereton, D. Budgen, M. Turner, J. Bailey, S. Linkman, Systematic literature reviews in software engineering—a systematic literature review, *Inf. Softw. Technol.* 51 (1) (2009) 7–15.
- [11] J.M. Corbin, A. Strauss, Grounded theory research: Procedures, canons, and evaluative criteria, *Qual. Sociol.* 13 (1) (1990) 3–21.
- [12] C. Wohlin, P. Runeson, M. Höst, M.C. Ohlsson, B. Regnell, A. Wesslén, *Experimentation in Software Engineering*, Springer Science & Business Media, 2012.
- [13] C. Wohlin, Guidelines for snowballing in systematic literature studies and a replication in software engineering, in: Proceedings of the 18th International Conference on Evaluation and Assessment in Software Engineering, 2014, pp. 1–10.
- [14] J.A. Russell, A circumplex model of affect, *J. Pers. Soc. Psychol.* 39 (6) (1980) 1161.
- [15] K.R. Scherer, What are emotions? And how can they be measured? *Soc. Sci. Inf.* 44 (4) (2005) 695–729.
- [16] S.H. Schwartz, An overview of the Schwartz theory of basic values, *Online Readings Psychol. Cult.* 2 (1) (2012) 0919–2307, <http://dx.doi.org/10.9707/2307-0919.1116>.
- [17] A. Razzaq, G. Botterweck, Q. Lai, J. Buckley, Empirical pathways to developer experience: A facet-based synthesis of empirical designs and guidelines, *J. Syst. Softw.* (2025) 112648, <http://dx.doi.org/10.1016/j.jss.2025.112648>, URL: <https://www.sciencedirect.com/science/article/pii/S0164121225003176>.
- [18] L. Gonçalves, K. Farias, B. da Silva, J. Fessler, Measuring the cognitive load of software developers: A systematic mapping study, in: 2019 IEEE/ACM 27th International Conference on Program Comprehension, ICPC, 2019, pp. 42–52, <http://dx.doi.org/10.1109/ICPC.2019.00018>.
- [19] W. Hussain, H. Perera, J. Whittle, A. Nurwidyanoro, R. Hoda, R.A. Shams, G. Oliver, Human values in software engineering: Contrasting case studies of practice, *IEEE Trans. Softw. Eng.* 48 (5) (2022) 1818–1833, <http://dx.doi.org/10.1109/TSE.2020.3038802>.
- [20] W.G. Parrott, *Emotions in Social Psychology: Essential Readings*, psychology Press, 2001.
- [21] D. Spencer, *Card Sorting: Designing Usable Categories*, Rosenfeld Media, 2009.
- [22] J. Sim, C.C. Wright, The kappa statistic in reliability studies: use, interpretation, and sample size requirements, *Phys. Ther.* 85 (3) (2005) 257–268.
- [23] A. Ampatzoglou, S. Bibi, P. Avgeriou, M. Verbeek, A. Chatzigeorgiou, Identifying, categorizing and mitigating threats to validity in software engineering secondary studies, *Inf. Softw. Technol.* 106 (2019) 201–230, <http://dx.doi.org/10.1016/j.infsof.2018.10.006>, URL: <https://www.sciencedirect.com/science/article/pii/S0950584918302106>.
- [24] P. Warr, The measurement of well-being and other aspects of mental health, *J. Occup. Psychol.* 63 (3) (1990) 193–210.